AGRIFLOW INDIA

Nationwide Agriculture Supply & Intelligence Network

Complete Product Requirements Document — Billion-Dollar Edition

Version 1.0 | Confidential & Proprietary

Architecture: Zero-INR Infrastructure · Serverless-First · Data-Intelligence Core

---

> **Vision Statement:** Build India's definitive agriculture supply discovery network — a living, breathing intelligence layer between farmers, FPOs, and buyers — that transforms raw crop production data into a national agricultural intelligence grid, creates price transparency, eliminates information asymmetry, and becomes the Bloomberg Terminal of Indian agriculture.

TABLE OF CONTENTS

SECTION 1 — EXECUTIVE SUMMARY & MARKET OPPORTUNITY

1.1 Product Identity

Field	Detail
Product Name	AgriFlow India
Tagline	*From Field to Intelligence. India's Agriculture Supply Network.*
Category	AgriTech · Supply Intelligence · Data Platform
Geography	India-wide (Phase 1: AP, Telangana, Karnataka, Maharashtra, Punjab)
Primary TAM	₹8,00,000 Cr+ Indian agriculture economy
Platform TAM (addressable)	₹45,000 Cr (agri-tech SaaS + supply chain intelligence)
Revenue Target (Yr 3)	₹150–₹400 Crores
Infrastructure Cost	₹0/month fixed (zero-INR serverless architecture)
Maintenance Model	Self-healing, event-driven, auto-scaling

1.2 The Problem — Five Dimensions of Agricultural Inefficiency

Problem 1: Invisible Supply Buyers — traders, exporters, processors, supermarkets — have zero visibility into what is growing, where it is, when it will be harvested, and in what volumes. They make sourcing decisions on phone calls, personal networks, and APMC rumor. This causes extreme price volatility, panic buying, and supply shocks.

Problem 2: Information Asymmetry for Farmers Farmers plant in isolation. They have no market signal. A tomato farmer in Kurnool and a tomato farmer in Nashik both plant tomatoes in the same season because neither knows the other exists. The result: simultaneous glut, price crash, and distress sales at ₹2/kg.

Problem 3: FPO Operational Blindness India has 10,000+ registered FPOs. Most operate manually — no procurement software, no inventory tracking, no aggregated supply reporting. They cannot leverage their collective volume because they cannot measure it.

Problem 4: No National Crop Production Intelligence India's government collects production data annually with 18-month delays. No real-time crop production intelligence exists. Policy, procurement, and pricing all happen on stale data. This is a ₹3,00,000 Cr+ problem.

Problem 5: Missing Market Connections for Small Farmers India has 150 million farming households. 85% are small and marginal farmers (< 2 hectares). They have no access to institutional buyers — exporters, processors, supermarkets. The entire supply chain intermediary cost (30–40%) is a tax on this information gap.

1.3 The AgriFlow Solution

A **three-sided data-driven marketplace** that:

1. Creates a **digital farmer network** with real-time crop production declarations
2. Provides **enterprise FPO software** to aggregate, track, and publish collective supply
3. Delivers a **buyer intelligence platform** with nationwide supply discovery and forecasting

The fundamental insight: AgriFlow is not a trading platform. It is an intelligence layer. By collecting crop production data at source (farmers declaring what they've planted and when they'll harvest), AgriFlow builds a live, granular picture of India's agricultural supply — becoming the intelligence infrastructure that drives better decisions across the entire value chain.

1.4 Market Opportunity — Quantified

Segment	India Market Size	AgriFlow Addressable	Yr-5 Revenue Potential
FPO SaaS (10,000+ FPOs)	₹5,000 Cr	2,000 FPOs × ₹36,000/yr	₹72 Crores

Segment	India Market Size	AgriFlow Addressable	Yr-5 Revenue Potential
Buyer Intelligence (5,000+ large buyers)	₹25,000 Cr	3,000 buyers × ₹30,000/yr	₹90 Crores
Agri Data Licensing (Govt, Banks, NBFCs)	₹8,000 Cr	50 institutions × ₹50L/yr	₹250 Crores
Farmer Subscriptions (50M farmers)	₹12,000 Cr	5M × ₹100/yr	₹50 Crores
Financial Services Referral (Loans, Insurance)	₹1,50,000 Cr	2% referral commission	₹200 Crores
Total Year 5 Potential			**₹662 Crores**

1.5 Why AgriFlow Becomes a ₹1,000 Crore Platform

1. **Data Network Effect:** Every farmer who joins makes the intelligence better for all buyers. Every buyer inquiry signals demand to all farmers. Classic two-sided network moat.
2. **Government Alignment:** India's Digital Agriculture Mission (2025–2030) mandates farmer digitization. AgriFlow is built precisely for this wave.
3. **No Operational Complexity:** No logistics, warehousing, or direct trading. Pure data and software — 85%+ gross margins.
4. **Sticky SaaS:** FPOs that build operations on AgriFlow don't switch. Data accumulation makes the product better over time.
5. **Regulatory Moat:** Becomes the preferred data partner for NABARD, SFAC, state agriculture departments. Government-endorsed network effects are near-unassailable.

SECTION 2 — PRODUCT VISION, MISSION & DESIGN PRINCIPLES

2.1 Vision

> *"To make every kilogram of Indian agriculture visible, predictable, and efficiently connected — from the smallest village farm to the largest export terminal."*

2.2 Mission

Build a frictionless digital infrastructure that a semi-literate farmer in Warangal can use in

Telugu with voice input, that an FPO secretary in Raichur can operate as full procurement software, and that a commodity trader in Mumbai can use to get real-time supply intelligence from 15 states — all running on zero fixed-cost infrastructure.

2.3 Core Design Principles

Principle	Implementation
Farmer First	Farmer app requires < 5MB, works on 2G, supports voice in vernacular
Data as Product	Every interaction generates intelligence; design for data quality
Zero Friction Onboarding	Farmer onboards in under 3 minutes with just a phone number
Privacy by Role	Strict access tiers: farmers see only their data; buyers see aggregate
Trust Infrastructure	Every data point traceable to source; verified via OTP and location
Offline Resilient	Core farmer data entry works without internet; syncs when online
Vernacular Native	Full support: Telugu, Kannada, Hindi, Marathi, Punjabi, Tamil
Lightweight by Default	No heavy frameworks on farmer app; React Native with minimal dependencies

---

SECTION 3 — BUSINESS MODEL & REVENUE ARCHITECTURE

3.1 Three-Tier Revenue Model

Tier 1: Farmer App Revenue (Volume, Low-ARPU)

Plan	Price	Target	Annual Revenue (Yr 3: 2M farmers)
Free	₹0	80% of farmers	₹0 (data is the product)
AgriPass	₹99/year	15% of farmers	₹30 Crores
AgriPass Pro	₹249/year	5% of farmers	₹25 Crores

AgriPass features: Priority buyer inquiries, yield analytics, soil health recommendations, weather alerts for their district, crop price history.

AgriPass Pro features: All of AgriPass + direct buyer connections, premium market price access, crop loan eligibility score, agronomist advisory access.

Philosophy: Maximize farmer adoption first. Data from 5 million farmers is worth ₹500 Crores in intelligence licensing. Never gate basic functionality behind payment.

Tier 2: FPO SaaS Revenue (Core B2B, Medium-ARPU)

Plan	Price/Month	Annual	Features
Starter	₹999	₹11,988	Up to 100 farmers, basic procurement, supply listing
Growth	₹2,499	₹29,988	Up to 500 farmers, full procurement, inventory, analytics
Enterprise	₹4,999	₹59,988	Unlimited farmers, API access, custom reports, priority support
State Cluster	₹14,999	₹1,79,988	Multi-FPO management, government reporting, NABARD integration

Revenue at scale: 3,000 FPOs on Growth plan = ₹90 Crores/year recurring.

Tier 3: Buyer Intelligence Revenue (High-ARPU, High-Margin)

Plan	Price/Year	Features
Explorer	₹9,999	5 states, crop search, 30-day forecasts
Trader	₹24,999	10 states, full supply discovery, inquiry system, alerts
Enterprise	₹59,999	All-India, API access, custom intelligence reports, dedicated analyst
Institutional	Custom (₹2L–₹20L)	Government agencies, banks, large exporters; raw data feeds

Revenue at scale: 2,000 buyers on Trader plan = ₹50 Crores/year. 50 institutional contracts = ₹50 Crores/year.

Tier 4: Data Licensing & API (Strategic, High-Margin)

Customer Type	Product	Annual Revenue
NABARD, RBI	Crop production intelligence feeds	₹2–₹5 Crores/license

Customer Type	Product	Annual Revenue
State Agriculture Departments	Real-time supply dashboards	₹50L–₹2 Crores/state
Crop Insurance Companies (AIC, etc.)	Farmer crop declaration data	₹1–₹3 Crores/company
Commodity Exchanges (NCDEX, MCX)	Supply forecasting signals	₹2–₹5 Crores/exchange
Food Processing Companies	Regional supply intelligence	₹25L–₹1 Crore/company

Tier 5: Financial Services Referral (Passive Revenue)

Service	Commission Model
Crop Loans (NABARD / SFBs)	0.5–1% of loan amount disbursed
Crop Insurance (PMFBY facilitation)	₹50–₹200/policy enrolled
Input Finance	2–4% commission on input credit facilitated
Agri Gold Loans	0.25% referral fee

3.2 Unit Economics

Metric	Value
Farmer CAC	₹15–₹30 (WhatsApp + FPO-led)
FPO CAC	₹500–₹1,500 (direct sales + govt partnerships)
Buyer CAC	₹2,000–₹8,000 (digital + field sales)
Gross Margin (SaaS)	85–92%
Gross Margin (Data Licensing)	95%+
Payback Period (FPO)	< 3 months
Payback Period (Buyer)	< 6 months

SECTION 4 — USER TYPES, PERSONAS & ROLE MATRIX

4.1 External User Personas

Persona 1: Raju — The Small Farmer

- **Profile:** 48 years old, 3.5 acres, Kurnool district, AP; grows groundnut and chilli
- **Device:** Android (₹6,000 Redmi), 4G with poor signal; uses WhatsApp daily
- **Language:** Telugu only; can read basic text; prefers voice
- **Pain:** Sells entire harvest to local mandi at distressed prices; no market visibility
- **Goal:** Know when buyers are coming, what price to expect, whether to harvest now or wait
- **Key App Need:** Voice input in Telugu; crop declaration in 3 steps; buyer inquiry notification

Persona 2: Krishnamurthy — The FPO Secretary

- **Profile:** 38 years old; secretary of "Rayalaseema Farmers Producer Company"; manages 450 member farmers
- **Device:** Android tablet + desktop; comfortable with apps; uses Excel for procurement records
- **Language:** Telugu + basic English
- **Pain:** Manual Excel procurement tracking; no inventory visibility; can't publish collective supply to buyers; farmer payment tracking is chaotic
- **Goal:** Digital procurement records, supply publishing to buyers, farmer payment tracking, NABARD reporting
- **Key App Need:** Full procurement flow; inventory management; supply listing; exportable reports

Persona 3: Vikram — The Commodity Trader

- **Profile:** 44 years old; based in Mumbai; buys tomato, onion, potato for processing plants; sources from AP, MH, HP
- **Device:** MacBook + iPhone; heavily data-driven; uses Bloomberg, WhatsApp for sourcing
- **Pain:** Calls 30 agents every day to understand supply; no reliable supply forecast; gets caught in supply shortfalls; over-buys when price signals are wrong
- **Goal:** See real-time crop availability across 5 states; get harvest forecasts 30 days out; contact verified FPOs directly
- **Key App Need:** Supply search engine with state/district/crop filters; 30-day harvest forecast; FPO contact system

Persona 4: Priya — The Export Manager

- **Profile:** 33 years old; Agro Export company, Bengaluru; exports fresh vegetables to UAE and Singapore
- **Device:** MacBook, iPhone; data-heavy user
- **Pain:** Supply commitments to international buyers require guaranteed volumes 60 days out; current sourcing is unreliable; no visibility into certified/quality-grade supply
- **Goal:** Identify certified (GlobalGAP / organic) supply ahead of export season; build direct FPO relationships; get quality grade data
- **Key App Need:** Quality-grade filtering; organic certification filter; advance harvest calendar; FPO direct contact

Persona 5: Suresh — The Food Processing Plant Head

- **Profile:** 55 years old; tomato paste factory in Pune; needs 500 tons/month tomato supply
- **Device:** Desktop + Android; uses enterprise ERP (SAP Lite)
- **Pain:** Seasonal supply gaps cause plant under-utilization; can't plan production schedule due to supply uncertainty; broker margins eat 20% of procurement cost
- **Goal:** Subscribe to real-time tomato supply intelligence; connect directly with FPOs; plan production 45 days in advance
- **Key App Need:** High-volume supply search; 45-day forecasts; supply trend charts; API integration with internal ERP

Persona 6: Meena — The State Agriculture Official

- **Profile:** 42 years old; Joint Director, AP Agriculture Department
- **Pain:** Crop production data is 18-months delayed; can't target subsidies or intervention efficiently; APMC data is incomplete
- **Goal:** Real-time district-wise crop production intelligence; early warning for surplus (price crash) or deficit (shortage)
- **Key App Need:** State-level intelligence dashboard; district heatmaps; harvest forecast calendar; exportable reports for policy

4.2 Internal Role Matrix

Role	Code	Level	Key Responsibilities
Platform Admin	PA	L6	Full system access; user management; data exports; configs
Data Quality Analyst	DQA	L4	Review and validate crop declarations; flag anomalies
FPO Success Manager	FSM	L3	Onboard FPOs; training; retention; upsell
Buyer Success Manager	BSM	L3	Onboard buyers; demo intelligence features; renewals
Farmer Field Agent	FFA	L2	Assist farmer onboarding in field; data accuracy verification
Support Agent	SA	L1	Tickets; WhatsApp support; escalations
Finance Admin	FA	L4	Payments; reconciliation; GST; invoicing
Data Science Ops	DSO	L5	ML model monitoring; intelligence pipeline; data quality

4.3 Permission Matrix

Action	Farmer	FPO Admin	FPO Staff	Buyer Explorer	Buyer Enterprise	Field Agent	Platform Admin
View own farm data	✅	❌	❌	❌	❌	✅	✅
View member farmer data	❌	✅	✅	❌	❌	❌	✅
View aggregated supply	❌	✅	✅	✅	✅	❌	✅
View individual farmer (named)	❌	✅ (members)	✅ (members)	❌	❌ (with consent)	✅	✅
Post crop declaration	✅	❌	❌	❌	❌	✅	✅

Action	Farmer	FPO Admin	FPO Staff	Buyer Explorer	Buyer Enterprise	Field Agent	Platform Admin
Post FPO supply listing	❌	✅	✅	❌	❌	❌	✅
Send buyer inquiry	❌	❌	❌	✅	✅	❌	✅
View intelligence dashboards	❌	✅ (limited)	❌	✅	✅	❌	✅
Export raw data	❌	✅ (own)	❌	❌	✅ (aggregate)	❌	✅
Access data API	❌	❌	❌	❌	✅	❌	✅
Verify KYC	❌	❌	❌	❌	❌	✅	✅

SECTION 5 — PLATFORM ARCHITECTURE — SYSTEM DESIGN

5.1 High-Level Architecture

```
┌─────────────────────────────────────────────────────────────────────┐
│                       CLIENT APPLICATIONS                             │
│                                                                       │
│  ┌─────────────────┐  ┌─────────────────────┐  ┌─────────────────────┐ │
│  │   FARMER APP    │  │ FPO MANAGEMENT      │  │ BUYER INTELLIGENCE│ │
│  │ React Native    │  │ PLATFORM            │  │ APP               │ │
│  │ (Expo)          │  │ Next.js Web App     │  │ Next.js Web App   │ │
│  │ iOS + Android   │  │ + React Native Tab  │  │ + React Native    │ │
│  │ Vernacular UI   │  │ (FPO field staff)   │  │ (iOS + Android)   │ │
│  └────────┬────────┘  └──────────┬──────────┘  └─────────┬─────────┘ │
└───────────┼──────────────────────┼───────────────────────┼───────────┘
            │                      │                       │
            └──────────────────────┼───────────────────────┘
                                   │ HTTPS / WSS
┌──────────────────────────────────▼──────────────────────────────────┐
│                    CLOUDFLARE EDGE NETWORK                           │
│          CDN · WAF · DDoS Protection · SSL · Rate Limiting           │
```

```
└──────────────────────────────────────┬──────────────────────────────────────┘
                                       │
                                       ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                       API GATEWAY LAYER                                      │
│               Supabase Edge Functions (Deno Runtime)                         │
│     Auth Middleware · Role-Based Access · Request Validation · Logging       │
└──────────┬──────────────────┬──────────────────┬─────────────────────────────┘
           │                  │                  │
           ▼                  ▼                  ▼
┌─────────────────┐  ┌─────────────────┐  ┌──────────────────────────┐
│  CORE DATABASE  │  │ REALTIME LAYER  │  │   INTELLIGENCE ENGINE    │
│  Supabase       │  │ Supabase        │  │   - pgvector (embeddings)│
│  PostgreSQL 15  │  │ Realtime        │  │   - Supply forecasting   │
│  + PostGIS      │  │ (Notifications, │  │   - Crop heatmaps        │
│  + pgvector     │  │  Alerts)        │  │   - Price signals        │
│  + Full-text    │  └─────────────────┘  │   - Anomaly detection    │
│  search         │                       └──────────────────────────┘
└─────────────────┘
           │
           ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                      BACKGROUND INTELLIGENCE PIPELINE                        │
│  Trigger.dev Jobs · Upstash QStash · Cron Tasks                              │
│  - Harvest date approaching alerts                                           │
│  - Supply forecast computation (nightly batch)                               │
│  - Buyer-supply match alerts                                                 │
│  - Data quality scoring                                                      │
│  - Community feed moderation                                                 │
└──────────────────────────────────────────────────────────────────────────────┘
           │
           ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                     EXTERNAL INTEGRATIONS                                    │
│  Government APIs: eNAM · Agmarknet · PM-KISAN · State Revenue Records        │
│  Financial: NABARD · SFAC · Crop Insurance APIs                              │
│  Communication: MSG91 · Meta WABA · Resend (Email)                           │
│  Weather: IMD · OpenWeatherMap                                               │
│  Maps: MapBox · Bhuvan (ISRO)                                                │
└──────────────────────────────────────────────────────────────────────────────┘
```

5.2 Data Flow Architecture

```
FARMER LAYER:
  Farmer declares crop → Crop Declaration stored → Triggers harvest calendar entry
  Harvest date approaches → Auto-post availability notice → Buyer alerts triggered
```

```
FPO LAYER:
  FPO procures from farmer → Procurement record created → Inventory updated
  FPO publishes aggregated supply → Supply listing goes live → Buyer discovery
enabled

INTELLIGENCE LAYER:
  All crop declarations → Aggregated by crop × district × harvest window
  FPO procurement data → Volume and quality intelligence added
  Supply trend computed → Forecast model updated → Buyer intelligence refreshed

BUYER LAYER:
  Buyer searches supply → Sees aggregated + FPO listings → Sends inquiry
  Inquiry → Routed to FPO or farmer (consent-gated) → Offline negotiation begins
```

SECTION 6 — PLATFORM 1: FARMER APP — COMPLETE SPECIFICATION

6.1 Farmer App Overview

The Farmer App is the **primary data collection instrument** of the entire platform. Its design must optimize for:

- Maximum farmer adoption (free, simple, vernacular)
- High-quality data entry (structured fields, voice assist, location auto-detection)
- Minimum friction (3-minute onboarding, works on 2G, offline-capable)

The Farmer App is a React Native (Expo) application supporting iOS and Android, with a Progressive Web App fallback for feature phones via WhatsApp Web.

6.2 Farmer Onboarding Flow

Step 1: Phone Verification

```
Screen: Enter 10-digit Indian mobile number
Action: OTP sent via MSG91 (primary) + Fast2SMS (fallback)
OTP validity: 5 minutes; resend after 60 seconds; max 5 attempts/hour
On verify: Account created → Basic profile screen
```

Step 2: Basic Profile

```
Fields:
  Name (text + voice input)
  Village (text + location auto-detect)
  Mandal / Taluka
  District (cascading from village)
  State (auto-detected)
  Primary Language (Telugu / Hindi / Kannada / Marathi / Punjabi / Tamil)

Data source: If user enables location → auto-detect village/district via reverse
geocoding (MapBox)
If location denied → cascading dropdown (State → District → Mandal → Village)
Village data source: Government village master list (Lok Dhaba API or local DB
copy)
```

Step 3: Farm Details

```
Required fields:
  Total land owned (acres / bigha / hectare — unit selected by region)
  Land type: Irrigated / Rain-fed / Mixed
  Primary crops (multi-select from crop list, searchable, vernacular)

Optional fields (prompted after basic registration):
  Soil type: Red / Black / Alluvial / Sandy / Loamy / Laterite
  Farming method: Conventional / Organic / Natural Farming / Zero Budget
  Organic certification: Yes (upload certificate) / In Process / No
  Irrigation type: Drip / Sprinkler / Canal / Borewell / Rain-fed / None
  Farm survey number(s) (text, optional — for land record linkage)
```

Step 4: Welcome + Quick Action

```
Completion screen:
  "Welcome to AgriFlow, [Name]!"
  Progress bar: "Your profile is 60% complete — add your crop to get buyer
inquiries"
  CTA: "Declare Your Current Crop" → navigates to crop declaration
  Skip option: "I'll do this later" → home dashboard
```

6.3 Farmer Home Dashboard

```
Top Section:
  Greeting: "Namaskaram, Raju! ☀️" (time-aware, vernacular)
  Weather widget: Today's weather for their district (IMD API)
  Crop health advisory: District-specific alert (e.g., "Yellow mosaic virus alert
in Kurnool — check your chilli crop")

Quick Actions (Icon Grid):
  🌱 Declare Crop          📢 Post Availability
  📬 My Inquiries          📊 My Harvest Calendar
  💬 Community Feed        💰 My Earnings (future)

Active Crops Panel:
  Card per declared crop:
    Crop name + icon
    Area planted
    Expected harvest (countdown: "18 days to harvest")
    Status: Declared / Available / Sold / Harvested
    Quick action: "Post Availability" (if harvest is near)

Recent Buyer Inquiries:
  List of last 5 inquiries with crop, buyer type, quantity asked
  "You have 3 new inquiries on your Tomato listing"

Bottom Navigation:
  🏠 Home  🌱 My Crops  📢 Availability  💬 Community  👤 Profile
```

6.4 Crop Declaration Module

6.4.1 Declare New Crop

```
Step 1: Select Crop
  Search with vernacular name (Telugu: "టమాటా" finds Tomato)
  Visual crop icons (photo-based)
  Recent crops shown first (personalized)
  Crop list: 200+ crops organized by category (Vegetables / Fruits / Cereals /
Pulses / Spices / Cash Crops / Oilseeds / Plantation)

Step 2: Crop Details
  Area under this crop (acres/bigha/cents — user's preferred unit)
  Planting date (date picker; defaults to today; allows past dates)
```

```
  Expected yield (quantity + unit: kg / quintal / ton)
  Expected harvest date (date picker)
  Variety / Cultivar (optional; text or select from common varieties)

Step 3: Farm Location
  Auto-detect farm location (GPS pin)
  Manual: Village/district (auto-filled from profile)
  Option: Draw farm boundary on map (for future precision agriculture)

Step 4: Confirm & Save
  Summary card shown
  Editable before confirm
  On save: Crop declaration stored → Harvest calendar updated → Data engine
notified
```

6.4.2 Crop Declaration Data Schema

json

```json
{
  "declarationId": "AF-DEC-2024-AP-00892",
  "farmerId": "farmer_uuid",
  "cropCode": "TOM",
  "cropName": "Tomato",
  "cropNameLocal": "టమాటా",
  "area": 2.5,
  "areaUnit": "acres",
  "plantingDate": "2024-11-15",
  "expectedYield": 12,
  "yieldUnit": "tons",
  "expectedHarvestDate": "2024-01-20",
  "harvestWindowDays": 14,
  "variety": "Hybrid NS 585",
  "farmLocation": {
    "village": "Palavalasa",
    "mandal": "Nandigama",
    "district": "Krishna",
    "state": "Andhra Pradesh",
    "latitude": 16.5234,
    "longitude": 80.2891,
    "geo_point": "POINT(80.2891 16.5234)"
  },
  "irrigationType": "drip",
  "farmingMethod": "conventional",
  "organicCertified": false,
  "status": "declared",
  "qualityGradeExpected": "A",
  "dataSource": "farmer_self",
  "verifiedByFpo": false,
  "verifiedByAgent": false,
  "createdAt": "2024-11-16T08:30:00Z",
  "updatedAt": "2024-11-16T08:30:00Z"
}
```

6.5 Crop Availability Posting

When Harvest Approaches (T-15 days):

```
System trigger: 15 days before expectedHarvestDate
Farmer receives notification: "Your Tomato harvest is in 15 days. Post
availability to get buyer inquiries."
```

Availability Posting Form:

```
Pre-filled from declaration (editable):
  Crop name
  Available quantity (kg / quintal / ton)
  Harvest date (confirm or update)
  Farm location (village/district)

Additional fields for listing:
  Quality grade: Premium / Grade A / Grade B / Mixed (photo upload option)
  Packaging preference: Loose / Crate / Sack (specify)
  Minimum order quantity (optional)
  Price expectation (optional): ₹__ per kg/quintal (helps buyers filter)
  Organic/certified: Toggle (pulls from profile)
  Availability window: "Available for __ days from harvest"

Privacy settings:
  "Who can see this listing?"
    → All buyers (recommended)
    → Verified buyers only
    → FPO members only (if farmer is FPO member)

Post → Listing goes live → Buyers alerted
```

Listing Expiry:

- Auto-expires 3 days after expectedHarvestDate + harvestWindowDays
- Farmer prompted to update: "Has your Tomato been sold? Mark as sold or extend listing"

6.6 Farmer Inquiry Management

```
When buyer sends inquiry:
  Farmer receives push notification + SMS
  In-app notification with: Buyer type (Trader / Exporter / Processor), Crop,
Quantity requested

Inquiry Card:
  Buyer type (never buyer name for free farmers — privacy)
  Crop and quantity interested in
  Location of buyer's operation
  Timestamp
  Action buttons: "I'm Interested" / "Not Available" / "Reply via FPO"
```

```
"I'm Interested" flow:
  Farmer's contact shared with buyer (if farmer's privacy setting allows)
  OR: Inquiry routed to farmer's FPO (if member)
  OR: Farmer's WhatsApp linked (if opted-in)

Note: For farmers NOT on a paid plan, buyer contact details are not revealed
directly.
The inquiry is logged and farmer can express interest, after which AgriFlow
facilitates connection.
```

6.7 Harvest Calendar

```
View: Monthly calendar with crop harvest windows shown as color bands
  Green: Upcoming harvest (next 30 days)
  Yellow: Harvest window open (current)
  Gray: Past harvests

Each calendar entry:
  Crop name + quantity
  Status: Declared / Available / Inquiry Received / Sold
  Quick actions: Post Availability / View Inquiries

Timeline view alternative:
  List of crops sorted by harvest date
  Days-to-harvest countdown
  Alerts for crops with no availability posting yet
```

6.8 Community Feed

```
Purpose: Knowledge sharing network among farmers in same region

Post types:
  🚨 Pest Alert: "Yellow mosaic virus spotted in Guntur — check your crops"
  💧 Irrigation Tip: "Drip irrigation saves 40% water for tomatoes"
  🌧 Weather: "Heavy rain expected next 3 days in Nalgonda — delay spraying"
  🌱 Crop Advice: "Best time to plant Rabi chilli in AP is October 15-November
15"
  💰 Price: "Onion selling at ₹28/kg in Kurnool APMC today"
  ❓ Question: "Anyone using biofungicide for late blight in potato?"

Feed filters:
```

```
  My district / My state / National
  By crop type
  By post type

Feed rules:
  Default: Show posts from same district + crops matching farmer's declared crops
  Farmer cannot see other farmers' crop quantity data via feed
  No phone numbers or personal contact sharing (auto-redacted)
  Moderation: AI + human moderation for misinformation

Post creation:
  Text (with voice-to-text in vernacular)
  Photo (optional)
  Tag: crop, district, type
  Anonymous option (identity protected, location shown as district only)
```

6.9 Farmer Profile & Settings

```
Profile sections:
  Personal info (name, photo, phone, village)
  Farm info (total land, crops, irrigation, soil type)
  Certifications (organic, GlobalGAP, FPO membership)
  FPO membership (linked if member)
  Language preference
  Privacy settings:
    Show contact to buyers: Yes / Only FPO / No
    Show on supply map: Yes / District level / No
  Notification settings:
    Buyer inquiries: Push + SMS / Push only / SMS only
    Price alerts: On / Off
    Weather alerts: On / Off
    Community alerts: On / Off
  Help & Support
  Language toggle (changes entire app UI)
```

SECTION 7 — PLATFORM 2: FPO MANAGEMENT PLATFORM — COMPLETE SPECIFICATION

7.1 FPO Platform Overview

The FPO Management Platform is a **full-featured SaaS application** designed for Farmer

Producer Organization secretaries, managers, and field staff. It serves as the operational backbone of FPO activity — replacing manual registers, Excel sheets, and WhatsApp chaos with a structured digital system.

Access: Web (Next.js — primary for desk use) + React Native mobile tab (for field staff procurement recording)

7.2 FPO Registration & Onboarding

FPO Profile Setup

```
Organization Details:
  FPO / FPC / Cooperative name
  Type: FPO / Cooperative / PACS / VCO / Agri Company
  Registration number (MCA21 / State Cooperative Dept)
  SFAC registration (if applicable)
  State + District + Block
  Office address
  Year of establishment
  Total member count (approximate)
  Primary crops handled

Contact Details:
  CEO / Secretary name
  Phone (OTP verified)
  Email
  WhatsApp number (for buyer communication)

Banking Details:
  Bank name
  Account number (encrypted)
  IFSC
  Account holder name (for payment routing — future)

Documents (upload):
  Registration certificate
  Board resolution (authorizing platform usage)
  Latest audited accounts (optional — for NABARD integration)

Verification:
  AgriFlow verifies registration certificate (auto OCR + manual review)
  SLA: 24 hours
  Upon verification: FPO goes live; can add farmers and list supply
```

7.3 Farmer Member Management

7.3.1 Add Farmer Members

```
Option 1: Individual Add
  Phone number → if farmer already on AgriFlow, profile auto-pulled
  If new: Name + village + land area entered by FPO staff → farmer invited via SMS

Option 2: Bulk Import
  CSV upload (template provided)
  Fields: Name, Phone, Village, Land Area, Primary Crops
  Validation: Duplicate phone check, district match check
  Import result: Success count, failure list with reasons

Option 3: QR Code Join
  FPO generates QR code
  Farmer scans → verifies OTP → auto-added to FPO member list
  Field agent can show QR at farmer meetings for mass onboarding
```

7.3.2 Farmer Member Directory

```
List view (filterable):
  Farmer name, village, land area, active crops, membership date, payment status

Individual farmer card:
  Profile info
  All crop declarations (with area, expected harvest, status)
  Procurement history (what FPO has bought from this farmer)
  Payment summary (total paid, pending)
  FPO-linked crop declarations (FPO can see member crop data)
  Contact (call, WhatsApp from within app)

Crop pattern analytics (per farmer):
  What they grow, which seasons, typical yield, quality history
  This builds individual farmer production profiles
```

7.4 Procurement Recording

7.4.1 Record Procurement (Core Feature)

```
When farmer delivers produce to FPO collection center:
```

```
Step 1: Select farmer
  Search by name or phone
  Or scan farmer's app QR code (instant select)

Step 2: Crop & Quantity
  Select crop (auto-suggested from farmer's declared crops)
  Quantity received (number + unit)
  Quality grade: Premium / Grade A / Grade B / Reject
  Moisture content (optional, numeric)
  Visual defect check: None / Minor / Major (dropdown + photo)

Step 3: Pricing
  Procurement price per unit (₹)
  Total amount auto-calculated
  Deductions (if any): Transport / Packaging / Commission
  Net payable to farmer: auto-calculated

Step 4: Confirm & Generate Receipt
  Summary screen
  Generate farmer receipt (printable / shareable via WhatsApp)
  Record saved → Inventory updated → Farmer payment pending entry created
```

7.4.2 Procurement Record Schema

json

```json
{
  "procurementId": "AF-PROC-2024-00567",
  "fpoId": "fpo_uuid",
  "farmerId": "farmer_uuid",
  "farmerName": "Raju Reddy",
  "cropCode": "TOM",
  "cropName": "Tomato",
  "quantityReceived": 2500,
  "quantityUnit": "kg",
  "qualityGrade": "A",
  "moistureContent": 92.5,
  "procurementPricePerUnit": 18,
  "priceUnit": "per_kg",
  "grossAmount": 45000,
  "deductions": {
    "transport": 500,
    "commission": 0,
    "other": 0
  },
  "netPayableToFarmer": 44500,
  "paymentStatus": "pending",
  "paidAt": null,
  "procurementDate": "2024-01-20T10:30:00Z",
  "collectionCenter": "Nandigama Main Center",
  "recordedBy": "fpo_staff_uuid",
  "linkedDeclarationId": "AF-DEC-2024-AP-00892",
  "receiptUrl": "...",
  "notes": ""
}
```


7.5 Inventory Management

7.5.1 Inventory Overview Dashboard

```
Cards showing per crop:
  Crop name + icon
  Total quantity in inventory (all locations)
  Locations: aggregated from all collection centers / cold storages
  Days since procurement (freshness indicator for perishables)
  Status: Fresh / Aging / Critical (color-coded by days)
  Quick action: "Create Supply Listing" / "Update Inventory"
```

7.5.2 Inventory Record Structure

```
Per storage location entry:
  Crop
  Quantity (current / original received)
  Quality grade
  Storage location: Name + type (collection center / cold storage / open yard)
  Storage temperature (for cold chain tracking)
  Entry date (date received into storage)
  Days in storage (auto-calculated)
  Freshness alert threshold (configurable by crop: tomato=3 days, onion=30 days)
  Linked procurement IDs (which purchases make up this inventory)
```

7.5.3 Inventory Adjustments

```
Add to inventory (non-procurement source):
  E.g., return from buyer, transfer from another FPO center
Reduce inventory:
  Sold (link to buyer sale record)
  Wastage (quantity + reason: spoilage / damage / other)
  Transfer to another location
  Donated / Government procurement (e-NaM, MSP procurement)

Full audit trail: Every quantity change logged with who, when, why
```

7.6 Aggregated Supply Listing (FPO → Buyer Visible)

7.6.1 Create Supply Listing

```
This is the FPO's "advertisement" to the buyer marketplace.

Pre-filled from inventory (editable):
  Crop
  Available quantity (can be less than full inventory)
  Quality grade
  Location: District + FPO name (exact address shown only to verified buyers)

Additional fields:
  Price expectation (₹ per unit — optional; "Negotiable" default)
  Harvest/delivery window: "Available for next __ days"
  Minimum order quantity
  Packaging available: Loose / Crate / Sack / Custom
```

```
  Certifications: Organic / Pesticide-residue tested / None
  Special notes (varieties, unique attributes)

Privacy on listing:
  FPO name: Always shown
  Exact quantity from inventory: Shown as "Available: ~80 Tons" (approximate)
  Farmer data: Never shown (aggregated only)

Listing duration:
  Default: 15 days (auto-expires)
  Renewable: 1-tap refresh
```

7.6.2 Supply Listing States

```
Draft → Pending Review (auto-review: < 2 hours) → Active →
Inquiry Received → Negotiating → Sold / Partial Sale / Expired / Relisted
```

7.7 Buyer Communication & Inquiry Handling

```
FPO receives buyer inquiry notification:
  Push + SMS + Email
  Inquiry detail: Buyer type, quantity needed, timeline, location

FPO inquiry dashboard:
  Inbox of all inquiries (sorted by: latest / quantity / match score)
  Inquiry card:
    Crop + quantity interested in
    Buyer type (Trader / Exporter / Processor / Retailer)
    Buyer location / operation city
    Inquiry timestamp
    Action: Reply / Decline / Forward to CEO

Reply flow:
  In-app message (text only initially)
  Option to share FPO's official WhatsApp for further negotiation
  AgriFlow does NOT manage pricing or deal terms
  Platform records that contact was made (for analytics)

Inquiry response tracking:
  % of inquiries responded within 24 hours (performance metric)
  Shown on FPO profile (affects trust score)
```

7.8 Payment Management (FPO ↔ Farmers)

7.8.1 Farmer Payments

```
Payment queue: List of all pending farmer payments
  Farmer name, crop, quantity, amount due, days pending

Batch payment:
  Select multiple farmers → generate payment batch
  Bank transfer instruction sheet (downloadable CSV for bank)
  Or: Integrate with NPCI NACH for direct transfers (Phase 2)

Payment recording:
  Mark each farmer as paid: date, mode (cash / bank transfer / UPI)
  System generates payment receipt for farmer (SMS + WhatsApp)
  Payment history: Full ledger per farmer

Outstanding report:
  Farmers not paid in > 7 days: highlighted
  Total outstanding amount
  Aging analysis: 0-7 days / 7-15 days / 15+ days
```

7.8.2 Buyer Payments (Future Phase)

```
When FPO completes a sale to a buyer:
  Record sale: Buyer, crop, quantity, price, sale date
  System tracks: Invoice generated / Payment received / Overdue
  Net settlement calculation:
    Gross sale revenue
    Less: Farmer procurement cost
    Less: FPO operating cost (configurable)
    Net profit / Commission
```

7.9 FPO Analytics Dashboard

```
Overview Cards (real-time):
  Total member farmers
  Active crop declarations (current season)
  Total inventory (all crops, all locations)
  Active supply listings
  Pending farmer payments
  Pending buyer inquiries
```

```
Season Report:
  Total procurement this season (by crop)
  Total value procured
  Average procurement price vs. APMC price (comparison)
  Quality grade breakdown
  Top 10 farmers by volume

Supply Intelligence (for FPO):
  Their crop supply vs. district total supply (% market share)
  Price trend for their key crops
  Demand signals: Which buyers are looking for their crops

Exportable Reports:
  Farmer-wise procurement statement (for NABARD/SFAC reporting)
  Crop-wise procurement and sale summary
  Farmer payment ledger
  Board meeting report (auto-generated PDF)
```

SECTION 8 — PLATFORM 3: BUYER INTELLIGENCE APP — COMPLETE SPECIFICATION

8.1 Buyer App Overview

The Buyer Intelligence App is the **monetization engine** of AgriFlow. It transforms the supply data collected from farmers and FPOs into actionable intelligence that allows buyers to source more efficiently, reduce cost, and eliminate supply shocks.

8.2 Buyer Onboarding

```
Step 1: Business Registration
  Company name
  Business type: Trader / Exporter / Food Processor / Retailer / Supermarket /
Government Agency / Research
  GSTIN (verified via GST API — free)
  Primary commodities of interest (multi-select, searchable)
  Sourcing regions (states — multi-select)
  Monthly sourcing volume (approximate: <10 Tons / 10-100 Tons / 100-1000 Tons /
1000+ Tons)

Step 2: Contact Person
```

```
  Name
  Designation
  Mobile (OTP verified)
  Email

Step 3: KYC
  GST certificate upload (auto-validated)
  PAN upload (for institutional contracts)
  Trade license (optional but boosts trust score)

Step 4: Plan Selection
  Show plan comparison: Explorer / Trader / Enterprise
  14-day trial for Trader plan (credit card not required)
  On trial: Access to 2 states of their choice
```

8.3 Buyer Home Dashboard

```
Top Stats:
  Active supply matches (crops matching their interest in their regions)
  New listings today
  Unread inquiries / responses
  Crops approaching harvest in their regions (next 10 days)

Supply Alerts Panel:
  "New: 45 Tons of Tomato available in Chittoor District — 8 days to harvest"
  "FPO Supply: 80 Tons of Onion in Nashik — Grade A — Available Now"
  "Price Alert: Tomato prices dropped 15% in AP this week"

Quick Actions:
  🔍 Search Supply        📋 My Watchlist
  📬 My Inquiries         📊 Intelligence Reports
  🔔 Alert Settings       📞 Saved Contacts

Navigation:
  🔍 Discover  📊 Intelligence  📋 Watchlist  💬 Inquiries  👤 Account
```

8.4 Supply Search Engine

8.4.1 Search Interface

```
Primary search: Text or crop selection
  Voice search (for mobile users)
```

```
  Smart suggest: "tomato" → "Tomato (Hybrid)" "Tomato (Cherry)" "Tomato (Desi)"

Filter panel (collapsible):
  Crop: Single or multi-select
  State: Multi-select (limited by plan)
  District: Cascading from state
  Harvest window: Slider (available in: 0 days / 1-7 days / 8-15 days / 15-30 days
/ 30-60 days)
  Quantity available: Min → max (tons)
  Quality grade: Premium / Grade A / Grade B / Any
  Source type: FPO Supply / Individual Farmer / Both
  Certification: Organic / Pesticide-free tested / Any
  Price expectation: Range (₹/quintal) — if listed
  Verified FPO only: Toggle

Sort options:
  Harvest date (soonest first)
  Quantity (highest first)
  Last updated
  Distance (from buyer's location)
  Match score (AI relevance)
```

8.4.2 Search Result Card — FPO Supply

```
┌─────────────────────────────────────────────────────────┐
│ 🍅 TOMATO — Grade A                    ✅ Verified FPO │
│                                                         │
│ Volume:   ~80 Tons                                      │
│ Location: Chittoor District, Andhra Pradesh             │
│ Harvest:  Available in 8 days (Jan 28, 2025)            │
│ FPO:      Chittoor Rythu FPC (120 members)              │
│ Price:    ₹18-₹22/kg (Negotiable)                       │
│ Packaging: Crate / Loose                                │
│                                                         │
│ [Send Inquiry]   [Add to Watchlist]   [View Details]   │
└─────────────────────────────────────────────────────────┘
```

8.4.3 Search Result Card — Aggregated Farmer Supply

```
┌─────────────────────────────────────────────────────────┐
│ 🧅 ONION — Mixed Grade                 📍 District Level │
│                                                         │
│ Volume:   ~150 Tons (aggregated, 23 farmers)            │
```

```
| Location: Kurnool District, Andhra Pradesh         |
| Harvest:  Next 15 Days                             |
| Source:   Individual Farmers (contact via AgriFlow) |
| Organic:  No                                       |
|                                                    |
| [Send District Inquiry]   [View Supply Trend]      |
└────────────────────────────────────────────────────┘
Note: Individual farmer contact available only with consent
```

8.5 Supply Detail View

```
FPO Supply Listing Detail:
  Full photos (of stored produce or field)
  FPO profile: Name, district, member count, years active, trust score
  Crop specifics: Variety, harvest date, quality grade description
  Quantity: Available now + forecasted in next 30 days
  Price history: Graph of this FPO's historical selling prices
  Certifications: Organic / Pesticide tested / None (with document links)
  FPO's response rate + average response time
  Previous buyer ratings (if any)
  Location map (district-level pin; exact address after inquiry accepted)
  Similar supply listings (recommended)

Farmer Aggregated Supply Detail (district-level):
  Crop, total volume, harvest window
  Number of farmers contributing (count, not names)
  Quality grade distribution (% Premium / Grade A / Grade B)
  Average farm size of contributors
  Price trend for this crop in this district (Agmarknet API)
  "Send district-level inquiry" → Routed to active FPOs in district or via
AgriFlow intermediary
```

8.6 Buyer Inquiry System

```
Send Inquiry Flow:
  Step 1: Crop, quantity needed (min), timeframe, delivery location
  Step 2: Special requirements (organic, grade, packaging, variety)
  Step 3: Price range (optional; "Best price" default)
  Step 4: Contact preference: Chat in-app / WhatsApp / Email
  Step 5: Confirm → Inquiry sent

Inquiry tracking:
```

```
  Sent → Delivered → Read → Replied / Declined / No response (48hr)

Inquiry dashboard:
  Status of all sent inquiries
  Quick re-send if no response in 48 hours
  Archive closed inquiries

For buyers: FPO's WhatsApp shared after FPO responds positively
  (Negotiation happens off-platform — AgriFlow tracks that connection was made)
```

8.7 Buyer Watchlist & Alerts

```
Watchlist (crop × region combinations):
  Example: "Tomato — AP" / "Onion — Maharashtra" / "Basmati Rice — Punjab"

Per watchlist item, buyer sets:
  Quantity threshold: Alert when > X tons available
  Harvest window: Alert when harvest within Y days
  Price threshold: Alert when price < ₹Z/unit
  New FPO supply: Alert immediately

Alert delivery:
  In-app push notification
  Email summary (daily digest or immediate)
  WhatsApp (if opted-in)

Smart alerts (AI-powered):
  "Based on your sourcing history, Chittoor district will have surplus Tomato in February — 60 days ahead"
  "Your usual Onion supplier in Nashik has reduced supply this season — explore Rajkot FPOs"
```

8.8 Supply Intelligence Reports (Buyer Exclusive Features)

Report 1: Crop Availability Heatmap

```
Visualization: India state map with color intensity = available supply
Filter: By crop, date range, quality grade
Drill-down: State → District → Village cluster
Data source: Aggregated declarations + FPO listings + historical actuals
Use case: "Where in India has the most Onion supply right now?"
```

Report 2: 30-Day Harvest Forecast

```
By crop × state:
  Timeline chart: Expected harvest volumes per 7-day window
  Confidence interval shown (based on declaration count vs. estimate)
  Comparison to last year's same period
  Weather impact overlay (IMD rainfall forecast)
Use case: "How much Tomato will be available in AP in the next 30 days?"
```

Report 3: Supply Trend Analysis

```
Historical supply trend per crop × region
  Price correlation (APMC data overlay from Agmarknet)
  Supply glut/deficit patterns by season
  Year-over-year comparison
Use case: "When should I buy Onion before prices spike?"
```

Report 4: Demand-Supply Gap Intelligence

```
Crops where declared supply > historical absorption (potential surplus — buy opportunity)
Crops where buyer inquiries exceed supply (deficit — act fast or plan alternatives)
Updated daily
Use case: "Which crops are under-supplied right now?"
```

Report 5: FPO Intelligence Report (Enterprise Only)

```
Directory of all active FPOs with:
  Crops handled, average volumes, quality score
  Historical reliability (response rate, deal completion rate)
  Location and connectivity details
  Contact initiation history (your company's previous interactions)
Use case: "Find the top 5 FPOs for organic tomato in AP"
```

SECTION 9 — CENTRAL AGRICULTURE DATA ENGINE

9.1 Data Engine Overview

The Central Agriculture Data Engine is the **intelligence core** of the platform. It is not a separate application — it is the data infrastructure and processing pipeline that sits underneath all three platforms.

What it ingests:

- Farmer crop declarations (what is planted, where, when, how much)
- FPO procurement records (what was actually harvested and sold, at what quality)
- FPO supply listings (what is available now)
- Buyer inquiry patterns (what buyers are actively seeking)
- APMC/eNAM price data (market prices from government sources)
- Weather data (district-level rainfall, temperature from IMD)
- Historical production data (government statistics as baseline)

What it produces:

- Aggregated supply intelligence by crop × district × harvest window
- Supply forecasting models (30/60/90 days)
- Price trend signals
- Supply-demand gap intelligence
- Crop production heatmaps
- Data feeds for government and institutional clients

9.2 Data Aggregation Logic

Supply Aggregation Rules

```
Level 1: Individual farmer declaration (never exposed to buyers)
Level 2: District-level aggregate (minimum 5 farmers threshold to avoid
identification)
  → Total volume, harvest window range, quality grade distribution
  → Shown on buyer discovery as "District-level supply"

Level 3: FPO listing (explicitly published by FPO — shown with FPO name)
  → Full detail visible to buyers with FPO branding

Level 4: State-level aggregate (for intelligence reports)
  → Total state supply by crop, trend analysis, forecasting

Privacy rule: No individual farmer identified in any buyer-visible surface.
```

```
Minimum aggregation: 5 farmers in any cell (if < 5, shown as "< X tons" without
count)
```

Forecast Model (Phase 2+)

```
Input signals:
  - Current season declarations (area × expected yield per crop × district)
  - Historical actual vs. declared yield (accuracy calibration)
  - Weather forecast impact factor (rainfall × temperature model)
  - Crop damage reports from community feed (input signal)
  - FPO procurement actuals (ground truth vs. forecast)

Output:
  - 30-day harvest forecast (by crop × district): volume estimate + confidence
range
  - 60-day outlook (lower confidence)
  - Season-end production estimate (state-level)

Model type: Statistical (moving average × weather correction) initially;
  ML model (XGBoost) after 2 seasons of data accumulation
```

9.3 Data Quality Engine

```
Every data point scored for quality:

Farmer declaration quality score (0-100):
  +20: Farmer is verified (Aadhaar-linked)
  +20: Farm location GPS-verified
  +15: Crop matches district's typical crops (Agmarknet crop map)
  +15: Yield per acre within ±50% of district average
  +10: FPO member (FPO can verify data)
  +10: Farmer has historic declarations (track record)
  +10: Photo attached (visual verification)
  -20: Yield or area claimed is statistical outlier (> 3 SD from district average)

Data shown as:
  High confidence (score 70+): Full data used in intelligence
  Medium confidence (score 40-69): Used but discounted in aggregates
  Low confidence (score <40): Flagged for review; excluded from intelligence

Quality score drives: Data agent field verification prioritization
```

9.4 Intelligence Pipeline (Data Flows)

```
Hourly batch:
  - Aggregate new declarations by crop × district
  - Update supply availability map
  - Trigger buyer alerts (new supply matches)

Daily batch:
  - Recompute 30-day harvest forecast
  - Update price trend signals (Agmarknet API pull)
  - Update weather overlay (IMD API)
  - Refresh intelligence dashboards
  - Compute FPO performance metrics
  - Data quality scoring of new declarations

Weekly batch:
  - Generate supply trend reports
  - Compute demand-supply gap analysis
  - Update FPO trust scores
  - Generate platform KPI report (internal)
  - Dispatch buyer weekly intelligence digest (email)

Seasonal (pre-season, post-season):
  - Season-end actual vs. forecast accuracy report
  - Calibrate forecast models
  - Generate government intelligence reports
  - NABARD / state agriculture dept data packages
```

SECTION 10 — DATA PRIVACY & ACCESS CONTROL MODEL

10.1 Data Classification

Data Type	Classification	Access
Farmer name, phone, Aadhaar	PII — Confidential	Farmer + FPO (member only) + Admin
Farmer's declared crop, area, location (village)	Semi-private	Farmer + FPO member + Aggregated for intelligence
Farmer's exact GPS coordinates	Private	Farmer + FPO + Field Agent

Data Type	Classification	Access
Aggregated district-level supply	Intelligence	All buyers (on subscription)
FPO supply listing (published)	Public intelligence	All buyers (on platform)
FPO procurement records	FPO-confidential	FPO + Admin
Buyer identity and inquiry history	Buyer-confidential	Buyer + Admin
Platform-level aggregate intelligence	Proprietary data asset	Licensed (government, institutions)

10.2 Row-Level Security Implementation

```sql
```

```
-- Farmers can only see their own data
CREATE POLICY "farmers_own_data" ON crop_declarations
  FOR ALL USING (farmer_id = auth.uid());

-- FPO staff can see their member farmers' data
CREATE POLICY "fpo_member_data" ON crop_declarations
  FOR SELECT USING (
    farmer_id IN (
      SELECT farmer_id FROM fpo_memberships
      WHERE fpo_id = (
        SELECT fpo_id FROM fpo_staff WHERE user_id = auth.uid()
      )
    )
  );

-- Buyers can only see aggregated supply listings (not individual declarations)
CREATE POLICY "buyers_see_aggregated_only" ON supply_listings
  FOR SELECT USING (
    listing_type IN ('fpo_listing', 'aggregated_district')
    AND status = 'active'
    AND EXISTS (
      SELECT 1 FROM user_subscriptions
      WHERE user_id = auth.uid()
      AND status = 'active'
      AND plan_type IN ('buyer_explorer', 'buyer_trader', 'buyer_enterprise')
    )
  );

-- Direct farmer contact only if farmer opted-in
CREATE POLICY "farmer_contact_consent" ON farmer_contact_permissions
  FOR SELECT USING (farmer_id = farmer_id AND consent_given = true);
```

10.3 Farmer Consent Management

```
Consent levels (farmer selects in settings):
  Level 0: No contact (data used only in aggregates — recommended minimum)
  Level 1: FPO contact only (only farmer's FPO can share contact with buyers)
  Level 2: Verified buyers may contact (buyer must be verified enterprise
subscriber)
  Level 3: All buyers may contact (farmer explicitly opt-in; gets more inquiries)

Consent withdrawal:
```

```
  Farmer can change consent at any time in settings
  Change takes effect immediately (no buffering)
  Existing active conversations: farmer can close

Buyer inquiry routing based on consent:
  Level 0: Buyer gets district-level aggregate; no direct contact
  Level 1: Inquiry routed to FPO secretary who can connect
  Level 2: Buyer gets FPO contact; FPO connects to farmer
  Level 3: Farmer's platform profile shared (WhatsApp opt-in handled separately)
```

SECTION 11 — USER JOURNEY MAPS — STEP-BY-STEP SCREEN FLOWS

Journey 1: Farmer Onboards and Declares Crop

```
[MINUTE 0-3: ONBOARDING]
Screen 1: Splash + Language Selection (Telugu / Hindi / Kannada / English)
Screen 2: "Enter your mobile number" → OTP sent
Screen 3: OTP entry (4-digit, auto-read on Android)
Screen 4: "What's your name?" (voice input supported)
Screen 5: Location permission request → auto-detect village/district
Screen 6: "How much land do you farm?" (slider + unit toggle)
Screen 7: "What do you grow?" (visual crop grid, multi-select)
Screen 8: Success: "Welcome to AgriFlow!" + dashboard

[MINUTE 3-7: FIRST CROP DECLARATION]
Screen 9: "Declare your current crop" CTA
Screen 10: Visual crop selector (large icons, voice search)
Screen 11: "How many acres of [Crop]?"
Screen 12: "When did you plant?" + "When do you expect to harvest?"
Screen 13: "How much do you expect to yield?"
Screen 14: Confirm screen: "Review your crop declaration"
Screen 15: Success: "Crop declared! You'll receive buyer inquiries near harvest
time."
Screen 16: Harvest calendar now shows the crop with countdown
```

Journey 2: FPO Records Procurement and Lists Supply

```
Screen 1: FPO Dashboard → "Record Procurement" CTA
Screen 2: "Select Farmer" → search or scan QR
```

```
Screen 3: Farmer selected → auto-shows their active crop declarations
Screen 4: Select crop → enter quantity, quality grade
Screen 5: Procurement price → deductions → net payable auto-calculated
Screen 6: Confirm → receipt generated → farmer notified via SMS
Screen 7: Inventory automatically updated
Screen 8: Dashboard shows new inventory entry

[Creating Supply Listing]
Screen 9: Inventory overview → "Create Supply Listing" on Tomato
Screen 10: Quantity, quality, price expectation, harvest window, notes
Screen 11: Preview how listing will appear to buyers
Screen 12: Publish → listing goes live on buyer marketplace
Screen 13: Buyer inquiries begin arriving → notification
```

Journey 3: Buyer Discovers Supply and Sends Inquiry

```
Screen 1: Buyer app home → supply alerts panel
Screen 2: "New: 80 Tons Tomato, Chittoor, AP" → tap to open
Screen 3: Supply detail: FPO profile, quantity, quality, price, location map
Screen 4: "Send Inquiry" → quantity needed, timeline, requirements
Screen 5: Inquiry sent → confirmation
Screen 6: FPO notified → FPO replies in-app
Screen 7: Buyer receives reply + FPO's WhatsApp shared
Screen 8: Negotiation moves to WhatsApp (off-platform)
Screen 9: Buyer marks inquiry as "Deal Done" (optional; helps analytics)

[Alternative: Search-led discovery]
Screen 2A: "Search Supply" → Crop: Tomato, State: AP, Harvest: Next 10 days
Screen 3A: Results (FPO listings + aggregated district supply)
Screen 4A: Filter: Grade A only, > 20 Tons
Screen 5A: 3 FPO results shown; tap to view detail
Screen 6A: Send inquiry → same as above
```

Journey 4: FPO Handles Buyer Inquiry

```
Screen 1: FPO dashboard → notification badge on Inquiries
Screen 2: Inquiry inbox: Buyer type, crop, quantity, timeframe
Screen 3: Open inquiry → buyer details (company type, volume, location)
Screen 4: "Reply" → in-app message
Screen 5: "Share WhatsApp" → buyer gets FPO's WhatsApp number
```

```
Screen 6: Mark inquiry as: Replied / Not Interested / Pending
Screen 7: Follow-up if no FPO response after 48 hours
```

SECTION 12 — TECHNICAL ARCHITECTURE — ZERO-INR STACK

12.1 Zero-INR Infrastructure Master Table

Layer	Service	Free Tier	When Cost Starts
Web Frontend	Next.js on Vercel	100GB bandwidth/month	After 100GB
Mobile	React Native (Expo)	Build server free	Play Store $25 one-time; App Store $99/yr
Database	Supabase PostgreSQL	500MB, 2GB bandwidth	>500MB ($25/mo)
Geospatial	PostGIS (in Supabase)	Included	—
Vector Search	pgvector (in Supabase)	Included	—
Full-text Search	Supabase FTS	Included	—
Auth	Supabase Auth	Unlimited	—
Realtime	Supabase Realtime	Unlimited connections	—
Edge Functions	Supabase Edge Fn	2M invocations/month	>2M
File Storage	Cloudflare R2	10GB, 0 egress cost	>10GB ($0.015/GB)
CDN	Cloudflare Free	Global CDN	—
Images	Cloudflare Images	100K transforms/month	—
Email	Resend	3,000 emails/month	>3K ($20/mo)
SMS	Fast2SMS	Pay-per-SMS (₹0.08)	Per use

Layer	Service	Free Tier	When Cost Starts
WhatsApp	Meta WABA	1,000 conversations/month	>1K
Maps	MapBox	50,000 loads/month	>50K
Weather	IMD Open API	Free (Government)	—
Crop Prices	Agmarknet API	Free (Government)	—
Village Master	Lok Dhaba	Free (Government)	—
GST Verify	GST API	Free	—
Land Records	State APIs (Dharani AP)	Free (Government)	—
PM-KISAN Data	PM-KISAN API	Free (Government)	—
Background Jobs	Trigger.dev	50K runs/month	>50K
Cache	Upstash Redis	10,000 commands/day	>10K
Queue	Upstash QStash	500 messages/day	>500
Monitoring	Sentry Free	5K errors/month	>5K
Analytics	PostHog Cloud	1M events/month	>1M
Push Notifs	Firebase FCM	Unlimited	—
CI/CD	GitHub Actions	2,000 min/month	>2K
Payments	Razorpay	0 fixed; 2%/txn	Per transaction
AI/ML	OpenAI API	Pay-per-use	Per token

Total Fixed Monthly Cost: ₹0 until platform reaches significant scale.

12.2 Tech Stack Decisions

```
Frontend Web (FPO Platform + Buyer App):
  Framework:    Next.js 14 (App Router, RSC)
```

```
  Language:     TypeScript 5
  Styling:      Tailwind CSS 3.4
  Charts/Maps:  Recharts + react-map-gl (MapBox)
  Components:   shadcn/ui + Radix UI
  State:        Zustand + TanStack Query (React Query)
  Forms:        React Hook Form + Zod
  Tables:       TanStack Table (for procurement/inventory)
  PDF:          React-PDF (for reports, receipts)
  Testing:      Vitest + Playwright

Mobile (Farmer App):
  Framework:    React Native + Expo SDK 51
  Navigation:   Expo Router
  State:        Zustand + MMKV (fast local storage)
  Camera:       expo-camera (for photo uploads)
  Location:     expo-location
  Notifications: expo-notifications + FCM
  Offline:      expo-sqlite (local data when offline)
  Voice Input:  expo-speech (TTS) + React Native Voice (STT)
  Vernacular:   i18n-js with Telugu/Hindi/Kannada/Marathi translations
  Testing:      Detox (E2E)

Backend:
  Runtime:      Deno (Supabase Edge Functions)
  Language:     TypeScript
  Validation:   Zod
  ORM:          Supabase JS + raw SQL for complex analytics
  Queue:        Trigger.dev (background jobs)
  Cache:        Upstash Redis

Data/Intelligence:
  Geospatial:   PostGIS (spatial queries)
  Vector:       pgvector (semantic search)
  FTS:          Supabase FTS (tsvector on crop names, localities)
  Analytics:    SQL aggregate queries → cached in Redis → served via API
  ML (Phase 2): Python microservice (Railway free tier) → scheduled batch
```

12.3 Offline Architecture (Farmer App Critical)

```
Problem: Farmers in rural areas have intermittent 2G/3G connectivity.
Solution: Offline-first architecture with sync-on-connect.

Local storage (expo-sqlite):
```

```
  - Farmer's own profile (always available offline)
  - Farmer's crop declarations (readable offline)
  - Community feed posts (last 50, cached)
  - Pending actions queue (declarations not yet synced)

Offline actions supported:
  ✅ Write crop declaration (queued for sync)
  ✅ Update availability posting (queued for sync)
  ✅ Write community post (queued for sync)
  ✅ View harvest calendar (from cache)
  ❌ Send/receive inquiries (requires connection)
  ❌ View live price data (requires connection)

Sync strategy:
  App checks connection on foreground → sync all pending
  Background sync every 30 minutes (if connected)
  Conflict resolution: Server timestamp wins for overlapping updates
  UI indicator: "Data synced 2 hours ago" or "Offline — 3 changes pending"
```

SECTION 13 — DATABASE SCHEMA & DATA MODELS

13.1 Complete Database Schema

```sql
```

```sql
-- ──────────────────────────────────────────────────────
-- USERS & AUTHENTICATION
-- ──────────────────────────────────────────────────────

CREATE TABLE users (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  phone                 VARCHAR(13) UNIQUE NOT NULL,
  email                 VARCHAR(255),
  name                  VARCHAR(150),
  role                  user_role_enum NOT NULL,
  preferred_language    language_enum DEFAULT 'telugu',
  profile_photo_url     TEXT,
  is_verified           BOOLEAN DEFAULT FALSE,
  is_active             BOOLEAN DEFAULT TRUE,
  kyc_level             SMALLINT DEFAULT 0, -- 0=phone, 1=aadhaar, 2=business
  aadhaar_hash          VARCHAR(64), -- SHA-256 (never store raw)
  pan_hash              VARCHAR(64),
  gstin                 VARCHAR(15),
  fcm_token             TEXT,
  metadata              JSONB DEFAULT '{}',
  created_at            TIMESTAMPTZ DEFAULT NOW(),
  last_active_at        TIMESTAMPTZ,
  updated_at            TIMESTAMPTZ DEFAULT NOW()
);

-- ──────────────────────────────────────────────────────
-- FARMER PROFILES
-- ──────────────────────────────────────────────────────

CREATE TABLE farmer_profiles (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id               UUID UNIQUE REFERENCES users(id) ON DELETE CASCADE,
  state                 VARCHAR(100) NOT NULL,
  district              VARCHAR(100) NOT NULL,
  mandal                VARCHAR(100),
  village               VARCHAR(150) NOT NULL,
  village_code          VARCHAR(20), -- LGD village code
  pincode               VARCHAR(6),
  latitude              DECIMAL(10,8),
  longitude             DECIMAL(11,8),
  geo_point             GEOGRAPHY(POINT,4326),
  total_land_acres      DECIMAL(10,3),
  land_unit             VARCHAR(20) DEFAULT 'acres', -- acres/bigha/cents
```

```sql
  irrigation_type      TEXT[], -- drip/sprinkler/canal/borewell/rain-fed
  farming_method       farming_method_enum DEFAULT 'conventional',
  soil_type            TEXT[],
  organic_certified    BOOLEAN DEFAULT FALSE,
  organic_cert_url     TEXT,
  organic_cert_expiry DATE,
  fpo_id               UUID REFERENCES fpo_profiles(id), -- primary FPO
  pm_kisan_id          VARCHAR(20), -- PM-KISAN farmer ID (if available)
  aadhaar_linked       BOOLEAN DEFAULT FALSE,
  contact_consent      contact_consent_enum DEFAULT 'fpo_only',
  survey_numbers       TEXT[], -- land survey numbers
  data_quality_score   SMALLINT DEFAULT 50,
  created_at           TIMESTAMPTZ DEFAULT NOW(),
  updated_at           TIMESTAMPTZ DEFAULT NOW()
);

-- ─────────────────────────────────────────────────────
-- FPO PROFILES
-- ─────────────────────────────────────────────────────

CREATE TABLE fpo_profiles (
  id                   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id              UUID UNIQUE REFERENCES users(id) ON DELETE CASCADE,
  fpo_name             VARCHAR(200) NOT NULL,
  fpo_type             fpo_type_enum NOT NULL, -- FPO/Cooperative/PACS/VCO
  registration_number VARCHAR(100) UNIQUE,
  sfac_id              VARCHAR(50),
  nabard_id            VARCHAR(50),
  state                VARCHAR(100) NOT NULL,
  district             VARCHAR(100) NOT NULL,
  block                VARCHAR(100),
  office_address       TEXT,
  latitude             DECIMAL(10,8),
  longitude            DECIMAL(11,8),
  ceo_name             VARCHAR(150),
  ceo_phone            VARCHAR(13),
  whatsapp_number      VARCHAR(13),
  email                VARCHAR(255),
  member_count         INTEGER DEFAULT 0,
  active_member_count INTEGER DEFAULT 0,
  primary_crops        TEXT[],
  bank_account_encrypted TEXT, -- AES-256 encrypted
  bank_ifsc            VARCHAR(11),
  bank_name            VARCHAR(100),
```

```sql
  subscription_plan   VARCHAR(50) DEFAULT 'free',
  subscription_expires_at TIMESTAMPTZ,
  verification_status verification_status_enum DEFAULT 'pending',
  verification_notes  TEXT,
  trust_score         SMALLINT DEFAULT 50,
  response_rate       DECIMAL(5,2), -- % of buyer inquiries responded to
  avg_response_hours  DECIMAL(6,2),
  year_established    SMALLINT,
  documents           JSONB DEFAULT '{}', -- {reg_cert: url, board_res: url}
  created_at          TIMESTAMPTZ DEFAULT NOW(),
  updated_at          TIMESTAMPTZ DEFAULT NOW()
);

-- ─────────────────────────────────────────────────────
-- FPO MEMBERSHIPS
-- ─────────────────────────────────────────────────────

CREATE TABLE fpo_memberships (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  fpo_id      UUID REFERENCES fpo_profiles(id),
  farmer_id   UUID REFERENCES farmer_profiles(id),
  joined_at   TIMESTAMPTZ DEFAULT NOW(),
  status      VARCHAR(20) DEFAULT 'active', -- active/inactive/left
  share_count INTEGER DEFAULT 0,
  UNIQUE(fpo_id, farmer_id)
);

-- ─────────────────────────────────────────────────────
-- BUYER PROFILES
-- ─────────────────────────────────────────────────────

CREATE TABLE buyer_profiles (
  id                 UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id            UUID UNIQUE REFERENCES users(id) ON DELETE CASCADE,
  company_name       VARCHAR(200) NOT NULL,
  business_type      buyer_type_enum NOT NULL,
  gstin              VARCHAR(15),
  pan                VARCHAR(10),
  state              VARCHAR(100),
  city               VARCHAR(100),
  sourcing_states    TEXT[],
  commodities        TEXT[], -- primary crops they buy
  monthly_volume_tons DECIMAL(10,2),
  subscription_plan buyer_plan_enum DEFAULT 'trial',
```

```sql
  subscription_expires_at TIMESTAMPTZ,
  api_key_hash        VARCHAR(64), -- for enterprise API access
  trial_started_at  TIMESTAMPTZ DEFAULT NOW(),
  contact_name        VARCHAR(150),
  contact_phone       VARCHAR(13),
  contact_email       VARCHAR(255),
  created_at          TIMESTAMPTZ DEFAULT NOW()
);

-- ──────────────────────────────────────────────────────
-- CROP MASTER
-- ──────────────────────────────────────────────────────

CREATE TABLE crops (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  crop_code         VARCHAR(10) UNIQUE NOT NULL, -- TOM, ONI, POT, etc.
  crop_name         VARCHAR(100) NOT NULL,
  crop_name_telugu VARCHAR(100),
  crop_name_hindi VARCHAR(100),
  crop_name_kannada VARCHAR(100),
  crop_name_marathi VARCHAR(100),
  category          crop_category_enum NOT NULL,
  sub_category      VARCHAR(50),
  typical_season    TEXT[], -- kharif/rabi/zaid
  typical_days_to_harvest INTEGER,
  yield_per_acre_typical_tons DECIMAL(6,3),
  unit_of_trade     VARCHAR(20), -- kg/quintal/ton
  is_perishable     BOOLEAN DEFAULT FALSE,
  shelf_life_days INTEGER, -- 0 if highly perishable
  major_states      TEXT[], -- states where commonly grown
  icon_url          TEXT,
  hsn_code          VARCHAR(10),
  created_at        TIMESTAMPTZ DEFAULT NOW()
);

-- ──────────────────────────────────────────────────────
-- CROP DECLARATIONS (Core Intelligence Data)
-- ──────────────────────────────────────────────────────

CREATE TABLE crop_declarations (
  id                        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  declaration_code          VARCHAR(30) UNIQUE NOT NULL,
  farmer_id                 UUID REFERENCES farmer_profiles(id),
  fpo_id                    UUID REFERENCES fpo_profiles(id), -- if FPO-assisted
```

```sql
  crop_id                  UUID REFERENCES crops(id),
  crop_code                VARCHAR(10),
  area                     DECIMAL(10,3) NOT NULL,
  area_unit                VARCHAR(20) DEFAULT 'acres',
  area_acres               DECIMAL(10,3), -- always normalized to acres
  planting_date            DATE NOT NULL,
  expected_harvest_date    DATE NOT NULL,
  harvest_window_days      SMALLINT DEFAULT 14,
  expected_yield           DECIMAL(10,3),
  yield_unit               VARCHAR(20) DEFAULT 'tons',
  yield_tons               DECIMAL(10,3), -- always normalized to tons
  variety                  VARCHAR(100),
  farming_method           farming_method_enum,
  organic_certified        BOOLEAN DEFAULT FALSE,
  irrigation_type          VARCHAR(50),
  -- Location (same as farmer profile unless different field)
  state                    VARCHAR(100) NOT NULL,
  district                 VARCHAR(100) NOT NULL,
  mandal                   VARCHAR(100),
  village                  VARCHAR(150) NOT NULL,
  latitude                 DECIMAL(10,8),
  longitude                DECIMAL(11,8),
  geo_point                GEOGRAPHY(POINT,4326),
  -- Status
  status                   declaration_status_enum DEFAULT 'declared',
  -- Data Quality
  data_quality_score       SMALLINT DEFAULT 50,
  verified_by_fpo          BOOLEAN DEFAULT FALSE,
  verified_by_agent        BOOLEAN DEFAULT FALSE,
  is_included_in_intelligence BOOLEAN DEFAULT TRUE, -- false if quality too low
  -- Tracking
  actual_yield_tons        DECIMAL(10,3), -- filled after harvest (ground truth)
  notes                    TEXT,
  photos                   TEXT[],
  created_at               TIMESTAMPTZ DEFAULT NOW(),
  updated_at               TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_crop_declarations_district ON crop_declarations(district, crop_code
CREATE INDEX idx_crop_declarations_harvest ON crop_declarations(expected_harvest_dat
CREATE INDEX idx_crop_declarations_geo ON crop_declarations USING GIST(geo_point);

-- ─────────────────────────────────────────────────────
-- AVAILABILITY LISTINGS (Farmer → Public)
```

```sql
-- ──────────────────────────────────────────────────────

CREATE TABLE availability_listings (
  id                   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_code         VARCHAR(30) UNIQUE NOT NULL,
  declaration_id       UUID REFERENCES crop_declarations(id),
  farmer_id            UUID REFERENCES farmer_profiles(id),
  crop_id              UUID REFERENCES crops(id),
  quantity_available   DECIMAL(10,3) NOT NULL,
  quantity_unit        VARCHAR(20) DEFAULT 'tons',
  harvest_date         DATE,
  available_from       DATE,
  available_until      DATE,
  quality_grade        quality_grade_enum,
  packaging            TEXT[],
  min_order_qty        DECIMAL(10,3),
  price_expectation    DECIMAL(10,2), -- per unit
  price_unit           VARCHAR(20),
  price_negotiable     BOOLEAN DEFAULT TRUE,
  organic_certified    BOOLEAN DEFAULT FALSE,
  certifications       TEXT[],
  visibility           listing_visibility_enum DEFAULT 'all_buyers',
  status               VARCHAR(30) DEFAULT 'active',
  notes                TEXT,
  photos               TEXT[],
  inquiry_count        INTEGER DEFAULT 0,
  created_at           TIMESTAMPTZ DEFAULT NOW(),
  expires_at           TIMESTAMPTZ
);

-- ──────────────────────────────────────────────────────
-- FPO SUPPLY LISTINGS (FPO → Buyer Marketplace)
-- ──────────────────────────────────────────────────────

CREATE TABLE fpo_supply_listings (
  id                   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_code         VARCHAR(30) UNIQUE NOT NULL,
  fpo_id               UUID REFERENCES fpo_profiles(id),
  crop_id              UUID REFERENCES crops(id),
  crop_code            VARCHAR(10),
  quantity_available   DECIMAL(10,3) NOT NULL,
  quantity_unit        VARCHAR(20) DEFAULT 'tons',
  quality_grade        quality_grade_enum,
  quality_description TEXT,
```

```sql
  harvest_from_date   DATE,
  harvest_to_date     DATE,
  available_from      DATE,
  available_until     DATE,
  state               VARCHAR(100) NOT NULL,
  district            VARCHAR(100) NOT NULL,
  storage_location    TEXT,
  price_expectation   DECIMAL(10,2),
  price_unit          VARCHAR(20),
  price_negotiable    BOOLEAN DEFAULT TRUE,
  min_order_qty       DECIMAL(10,3),
  packaging_available TEXT[],
  certifications      TEXT[],
  special_notes       TEXT,
  photos              TEXT[],
  linked_inventory_ids UUID[],
  status              VARCHAR(30) DEFAULT 'active',
  inquiry_count       INTEGER DEFAULT 0,
  view_count          INTEGER DEFAULT 0,
  created_at          TIMESTAMPTZ DEFAULT NOW(),
  expires_at          TIMESTAMPTZ,
  updated_at          TIMESTAMPTZ DEFAULT NOW()
);

-- ─────────────────────────────────────────────────────
-- PROCUREMENT RECORDS (FPO from Farmers)
-- ─────────────────────────────────────────────────────

CREATE TABLE procurement_records (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  procurement_code      VARCHAR(30) UNIQUE NOT NULL,
  fpo_id                UUID REFERENCES fpo_profiles(id),
  farmer_id             UUID REFERENCES farmer_profiles(id),
  declaration_id        UUID REFERENCES crop_declarations(id),
  crop_id               UUID REFERENCES crops(id),
  quantity_received     DECIMAL(10,3) NOT NULL,
  quantity_unit         VARCHAR(20) DEFAULT 'kg',
  quantity_tons         DECIMAL(10,3),
  quality_grade         quality_grade_enum,
  moisture_content      DECIMAL(5,2),
  defect_level          VARCHAR(30),
  procurement_price     DECIMAL(10,2) NOT NULL, -- per unit
  price_unit            VARCHAR(20),
  gross_amount          DECIMAL(12,2) NOT NULL,
```

```sql
  deduction_transport   DECIMAL(10,2) DEFAULT 0,
  deduction_commission  DECIMAL(10,2) DEFAULT 0,
  deduction_other       DECIMAL(10,2) DEFAULT 0,
  net_payable           DECIMAL(12,2) NOT NULL,
  payment_status        payment_status_enum DEFAULT 'pending',
  paid_amount           DECIMAL(12,2) DEFAULT 0,
  paid_at               TIMESTAMPTZ,
  payment_mode          VARCHAR(30), -- cash/upi/bank_transfer
  collection_center     VARCHAR(200),
  procurement_date      TIMESTAMPTZ NOT NULL,
  receipt_url           TEXT,
  recorded_by           UUID REFERENCES users(id),
  notes                 TEXT,
  created_at            TIMESTAMPTZ DEFAULT NOW()
);

-- ─────────────────────────────────────────────────────
-- FPO INVENTORY
-- ─────────────────────────────────────────────────────

CREATE TABLE fpo_inventory (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  fpo_id                UUID REFERENCES fpo_profiles(id),
  crop_id               UUID REFERENCES crops(id),
  storage_location      VARCHAR(200) NOT NULL,
  storage_type          VARCHAR(50), -- warehouse/cold_storage/open_yard
  quantity_current      DECIMAL(10,3) NOT NULL,
  quantity_unit         VARCHAR(20) DEFAULT 'tons',
  quality_grade         quality_grade_enum,
  temperature           DECIMAL(5,2), -- celsius (for cold chain)
  entry_date            DATE NOT NULL,
  days_in_storage       INTEGER GENERATED ALWAYS AS (CURRENT_DATE - entry_date) STORED
  freshness_status      VARCHAR(20), -- fresh/aging/critical/expired
  linked_procurement_ids UUID[],
  notes                 TEXT,
  last_verified_at      TIMESTAMPTZ,
  created_at            TIMESTAMPTZ DEFAULT NOW(),
  updated_at            TIMESTAMPTZ DEFAULT NOW()
);

-- ─────────────────────────────────────────────────────
-- BUYER INQUIRIES
-- ─────────────────────────────────────────────────────
```

```sql
CREATE TABLE buyer_inquiries (
  id                   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  inquiry_code         VARCHAR(30) UNIQUE NOT NULL,
  buyer_id             UUID REFERENCES buyer_profiles(id),
  inquiry_type         VARCHAR(30), -- fpo_listing/district_aggregate/farmer_direct
  target_fpo_id        UUID REFERENCES fpo_profiles(id),
  target_farmer_id     UUID REFERENCES farmer_profiles(id),
  listing_id           UUID, -- fpo_supply_listing or availability_listing
  crop_id              UUID REFERENCES crops(id),
  quantity_needed      DECIMAL(10,3),
  quantity_unit        VARCHAR(20),
  required_by_date     DATE,
  delivery_location    TEXT,
  quality_grade_req    quality_grade_enum,
  certifications_req   TEXT[],
  price_range_min      DECIMAL(10,2),
  price_range_max      DECIMAL(10,2),
  special_requirements TEXT,
  status               inquiry_status_enum DEFAULT 'sent',
  response_received_at TIMESTAMPTZ,
  contact_shared       BOOLEAN DEFAULT FALSE,
  deal_outcome         VARCHAR(30), -- deal_done/negotiating/declined/no_response
  notes                TEXT,
  created_at           TIMESTAMPTZ DEFAULT NOW(),
  updated_at           TIMESTAMPTZ DEFAULT NOW()
);

-- ─────────────────────────────────────────────────────
-- SUPPLY INTELLIGENCE (Computed Aggregates)
-- ─────────────────────────────────────────────────────

CREATE TABLE supply_intelligence (
  id                     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  crop_id                UUID REFERENCES crops(id),
  crop_code              VARCHAR(10),
  state                  VARCHAR(100),
  district               VARCHAR(100),
  intelligence_date      DATE NOT NULL, -- which date this snapshot is for
  forecast_window_days   SMALLINT, -- 7/14/30/60/90
  -- Supply data
  total_declared_area_acres DECIMAL(12,3),
  total_declared_yield_tons DECIMAL(12,3),
  farmer_count           INTEGER, -- number of farmers declaring (never exposed < 5)
  below_threshold        BOOLEAN DEFAULT FALSE, -- true if < 5 farmers (hide count)
```

```sql
  -- Forecast
  forecast_harvest_tons DECIMAL(12,3), -- adjusted for typical loss factors
  forecast_confidence   DECIMAL(5,2), -- 0-100%
  -- Quality distribution
  pct_grade_premium     DECIMAL(5,2),
  pct_grade_a           DECIMAL(5,2),
  pct_grade_b           DECIMAL(5,2),
  -- FPO listing data
  fpo_listed_tons       DECIMAL(12,3), -- tons in active FPO listings
  active_fpo_count      INTEGER,
  -- Price intelligence
  price_agmarknet       DECIMAL(10,2), -- latest APMC price (Agmarknet)
  price_trend_7d        DECIMAL(5,2), -- % change in 7 days
  price_trend_30d       DECIMAL(5,2),
  -- Demand signals
  active_buyer_inquiries INTEGER,
  -- Metadata
  data_quality_avg      DECIMAL(5,2),
  computed_at           TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(crop_code, state, district, intelligence_date, forecast_window_days)
);

CREATE INDEX idx_supply_intel_crop_district ON supply_intelligence(crop_code, distri

-- ─────────────────────────────────────────────────────
-- COMMUNITY FEED POSTS
-- ─────────────────────────────────────────────────────

CREATE TABLE community_posts (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  author_id      UUID REFERENCES users(id),
  post_type      post_type_enum NOT NULL,
  title          VARCHAR(200),
  content        TEXT NOT NULL,
  content_telugu TEXT,
  photos         TEXT[],
  crop_tags      TEXT[], -- crop codes this relates to
  state          VARCHAR(100),
  district       VARCHAR(100),
  is_anonymous   BOOLEAN DEFAULT FALSE,
  is_verified    BOOLEAN DEFAULT FALSE, -- marked by admin/agronomist
  upvotes        INTEGER DEFAULT 0,
  view_count     INTEGER DEFAULT 0,
  status         VARCHAR(20) DEFAULT 'published',
```

```sql
  moderation_flag BOOLEAN DEFAULT FALSE,
  created_at     TIMESTAMPTZ DEFAULT NOW()
);

-- ─────────────────────────────────────────────────────
-- SUBSCRIPTIONS
-- ─────────────────────────────────────────────────────

CREATE TABLE subscriptions (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID REFERENCES users(id),
  plan_code       VARCHAR(50) NOT NULL,
  plan_type       VARCHAR(30) NOT NULL, -- farmer/fpo/buyer
  status          sub_status_enum DEFAULT 'active',
  amount_paid     INTEGER, -- in paise
  starts_at       TIMESTAMPTZ NOT NULL,
  expires_at      TIMESTAMPTZ NOT NULL,
  payment_id      VARCHAR(100), -- Razorpay payment ID
  auto_renew      BOOLEAN DEFAULT FALSE,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- ─────────────────────────────────────────────────────
-- PAYMENTS
-- ─────────────────────────────────────────────────────

CREATE TABLE payments (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payment_code        VARCHAR(30) UNIQUE NOT NULL,
  payer_id            UUID REFERENCES users(id),
  payment_type        VARCHAR(50) NOT NULL,
  amount              INTEGER NOT NULL, -- paise
  currency            VARCHAR(3) DEFAULT 'INR',
  status              payment_status_enum DEFAULT 'initiated',
  gateway             VARCHAR(20) DEFAULT 'razorpay',
  gateway_order_id    VARCHAR(100),
  gateway_payment_id  VARCHAR(100) UNIQUE,
  gateway_signature   TEXT,
  subscription_id     UUID REFERENCES subscriptions(id),
  invoice_url         TEXT,
  created_at          TIMESTAMPTZ DEFAULT NOW(),
  updated_at          TIMESTAMPTZ DEFAULT NOW()
);
```

```sql
-- ─────────────────────────────────────────────────────
-- WATCHLISTS (Buyer Alerts)
-- ─────────────────────────────────────────────────────

CREATE TABLE buyer_watchlists (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  buyer_id              UUID REFERENCES buyer_profiles(id),
  crop_id               UUID REFERENCES crops(id),
  states                TEXT[],
  districts             TEXT[],
  min_quantity_tons     DECIMAL(10,3),
  max_harvest_days      SMALLINT,
  max_price_per_unit    DECIMAL(10,2),
  quality_grade_min     quality_grade_enum,
  certifications_req    TEXT[],
  alert_push            BOOLEAN DEFAULT TRUE,
  alert_email           BOOLEAN DEFAULT TRUE,
  alert_whatsapp        BOOLEAN DEFAULT FALSE,
  is_active             BOOLEAN DEFAULT TRUE,
  last_alerted_at       TIMESTAMPTZ,
  created_at            TIMESTAMPTZ DEFAULT NOW()
);

-- ─────────────────────────────────────────────────────
-- SUPPORT TICKETS
-- ─────────────────────────────────────────────────────

CREATE TABLE support_tickets (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ticket_code       VARCHAR(20) UNIQUE NOT NULL,
  user_id           UUID REFERENCES users(id),
  user_role         user_role_enum,
  category          ticket_category_enum,
  priority          ticket_priority_enum DEFAULT 'medium',
  status            ticket_status_enum DEFAULT 'open',
  subject           VARCHAR(200),
  description       TEXT,
  attachments       TEXT[],
  channel           VARCHAR(20), -- app/whatsapp/email
  assigned_to       UUID REFERENCES users(id),
  sla_deadline      TIMESTAMPTZ,
  resolved_at       TIMESTAMPTZ,
  rating            SMALLINT,
```

```
  created_at       TIMESTAMPTZ DEFAULT NOW()
);
```

SECTION 14 — API DESIGN & CONTRACT

14.1 API Structure

```
Base URL: https://api.agriflow.in/v1
Auth: Bearer JWT (Supabase)
Content-Type: application/json
Rate Limits: Enforced by Cloudflare + Upstash Redis
```

14.2 Core Endpoints

```yaml
```

```
# AUTH
POST   /auth/otp/send              # Send OTP (phone, role hint)
POST   /auth/otp/verify            # Verify OTP → JWT returned
POST   /auth/refresh               # Refresh JWT

# FARMER
GET    /farmer/profile             # Get farmer profile
POST   /farmer/profile             # Create farmer profile (post-auth)
PATCH  /farmer/profile             # Update profile

# CROP DECLARATIONS
POST   /declarations               # Create crop declaration
GET    /declarations               # Get my declarations
GET    /declarations/:id           # Get single declaration
PATCH  /declarations/:id           # Update declaration
PATCH  /declarations/:id/status # Update status (harvested/cancelled)

# AVAILABILITY LISTINGS (Farmer)
POST   /farmer/listings            # Post availability
GET    /farmer/listings            # Get my listings
PATCH  /farmer/listings/:id        # Update listing

# COMMUNITY FEED
GET    /feed                       # Get feed posts (district/state/national)
POST   /feed                       # Create post
POST   /feed/:id/upvote            # Upvote post
POST   /feed/:id/report            # Report post

# FPO
GET    /fpo/profile                # Get FPO profile
POST   /fpo/profile                # Create FPO profile
PATCH  /fpo/profile                # Update FPO profile

# FPO MEMBER MANAGEMENT
GET    /fpo/members                # Get member farmers list
POST   /fpo/members                # Add member farmer
DELETE /fpo/members/:farmerId      # Remove member
POST   /fpo/members/bulk-import # Bulk CSV import
GET    /fpo/members/invite-qr      # Generate join QR code

# PROCUREMENT
POST   /fpo/procurement            # Record procurement
GET    /fpo/procurement            # List procurement records
```

```
GET    /fpo/procurement/:id     # Single record
PATCH  /fpo/procurement/:id/payment # Record payment to farmer

# FPO INVENTORY
GET    /fpo/inventory           # Get inventory overview
POST   /fpo/inventory           # Add inventory entry
PATCH  /fpo/inventory/:id       # Update (adjust quantity, wastage)

# FPO SUPPLY LISTINGS
POST   /fpo/supply-listings     # Create supply listing
GET    /fpo/supply-listings     # My supply listings
PATCH  /fpo/supply-listings/:id # Update listing
DELETE /fpo/supply-listings/:id # Remove listing

# BUYER
GET    /buyer/profile           # Get buyer profile
POST   /buyer/profile           # Create buyer profile
PATCH  /buyer/profile           # Update

# SUPPLY DISCOVERY (Buyer)
GET    /supply/search           # Search supply listings
GET    /supply/:id              # Get listing detail
GET    /supply/intelligence     # Get intelligence for crop × district
GET    /supply/forecast         # Get harvest forecast
GET    /supply/heatmap          # Get crop heatmap data (GeoJSON)
GET    /supply/trends           # Price + supply trends

# BUYER INQUIRIES
POST   /inquiries               # Send inquiry
GET    /inquiries               # My inquiries (buyer)
GET    /inquiries/received      # Received inquiries (FPO/farmer)
PATCH  /inquiries/:id/respond   # Respond to inquiry
PATCH  /inquiries/:id/outcome   # Mark deal outcome

# WATCHLISTS & ALERTS
GET    /watchlists              # Get my watchlists
POST   /watchlists              # Create watchlist
PATCH  /watchlists/:id          # Update watchlist
DELETE /watchlists/:id          # Delete watchlist

# SUBSCRIPTIONS
GET    /plans                   # Get available plans
POST   /subscriptions           # Purchase subscription
GET    /subscriptions/current   # Active subscription
```

```
# PAYMENTS
POST   /payments/order           # Create Razorpay order
POST   /payments/verify          # Verify payment + activate subscription
GET    /payments/history         # Payment history

# INTELLIGENCE (High-value endpoints)
GET    /intelligence/district/:district/crop/:crop  # District supply intelligence
GET    /intelligence/state/:state                   # State-level summary
GET    /intelligence/national                       # National supply overview (ente
GET    /intelligence/fpo-directory                  # FPO directory with supply data

# GOVERNMENT / INTEGRATIONS
GET    /market-prices/:crop/:district  # Agmarknet price data
GET    /weather/:district              # IMD weather for district
GET    /crop-calendar/:district        # Typical crop calendar for district

# ADMIN (Internal)
GET    /admin/declarations/flagged     # Low quality data flags
PATCH  /admin/declarations/:id/verify  # Verify declaration
GET    /admin/fpo/pending              # FPOs awaiting verification
PATCH  /admin/fpo/:id/verify           # Verify FPO
GET    /admin/intelligence/pipeline    # Pipeline status

# WEBHOOKS
POST   /webhooks/razorpay              # Payment webhook handler
```

SECTION 15 — SECURITY ARCHITECTURE

15.1 Authentication Security

```
Multi-layer authentication:
  Layer 1: Phone OTP (mandatory all users)
  Layer 2: Aadhaar OTP link (optional, boosts KYC level)
  Layer 3: Business GST verification (buyers)
  Layer 4: TOTP (Google Authenticator) — internal admins only

Session management:
  Access token: 1-hour expiry
  Refresh token: 30-day, rotated on use
```

```
  Concurrent sessions: 3 devices max (farmer); 10 devices (FPO enterprise)
  Force logout: Admin-triggered invalidation
```

15.2 Data Security

```
PII Protection:
  Farmer phone: Only shown to FPO of their membership (never to buyers)
  Aadhaar: Stored as SHA-256 hash only; raw number never persisted
  Bank details: AES-256-GCM encrypted at application layer before storage
  GPS coordinates: Rounded to 0.01 degree for buyer-visible surfaces (~1km
precision)

Aggregation Privacy (k-Anonymity):
  Minimum k=5 farmers in any district × crop cell before showing data
  If < 5 farmers, shown as "< X tons" without count
  This prevents reverse-identification of individual farmers

RLS Policies: All tables have Supabase RLS enabled (described in Section 10)

Encryption:
  At rest: Supabase AES-256, Cloudflare R2 server-side encryption
  In transit: TLS 1.3 minimum; HSTS enforced
  Application layer: Bank details, sensitive fields (AES-256-GCM + key in Supabase
Vault)
```

15.3 API Security

```
Rate Limits (per endpoint category):
  OTP send: 5/hour per phone
  Search API: 100/minute per buyer
  Intelligence API: 20/minute (enterprise); 5/minute (trader)
  Declaration create: 50/day per farmer
  File upload: 100MB/day per user

Input Validation:
  All inputs: Zod schema validation on edge function
  SQL: Supabase client (parameterized queries always)
  File uploads: MIME type check + size limit + ClamAV scan
  Phone numbers: Regex /^[6-9]\d{9}$/

Bot Prevention:
  Cloudflare Bot Management (free tier): Challenge suspicious IPs
```

```
  CAPTCHA: On registration (invisible hCaptcha — free)
  Behavioral analysis: Upstash rate limit per IP + per user

Data Scraping Prevention:
  Intelligence API: Rate limited; enterprise requires API key
  Search results: Limited page size (20/page); deep pagination gated
  No bulk export endpoints for supply data (only own organization data)
```

SECTION 16 — UI/UX DESIGN SYSTEM

16.1 Design Philosophy

Three distinct design languages for three distinct user groups:

Farmer App: "Village market on your phone"

- Warm, earthy palette (saffron, green, brown)
- Large tap targets (60px minimum) — for calloused hands
- Photo-heavy UI — farmer recognizes crops visually, not by text
- Minimal text — maximum visual communication
- Voice primary — text secondary

FPO Platform: "Professional SaaS — not enterprise boring"

- Clean, data-dense layout for desktop use
- Green as primary (agriculture brand) with neutral grays
- Complex tables, charts, forms — clearly organized
- WhatsApp-like familiarity in communication modules

Buyer Intelligence: "Bloomberg meets Google Analytics"

- Dark-mode default — data-heavy environment
- High information density — traders want everything visible
- Chart-first interface — numbers in context of trends
- Premium feel — justifies ₹50,000/year subscription

16.2 Design Tokens

```css
```

```css
/* FARMER APP TOKENS */
:root[data-app="farmer"] {
  --color-primary:     #2d9b47;  /* AgriGreen */
  --color-secondary:   #e07b00;  /* Harvest Orange */
  --color-accent:      #f5c842;  /* Sunflower Yellow */
  --color-background:  #fefdf8;  /* Warm White (off-white) */
  --color-surface:     #ffffff;
  --color-text:        #1a1a1a;
  --font-display:      'Hind', 'Noto Sans Telugu', sans-serif; /* Works for Telugu *
  --font-body:         'Hind', 'Noto Sans', sans-serif;
  --radius-default:    16px;    /* Generous radius — friendly feel */
  --tap-target-min:    60px;    /* Accessibility for farmers */
}

/* FPO PLATFORM TOKENS */
:root[data-app="fpo"] {
  --color-primary:     #16a34a;  /* Professional Green */
  --color-secondary:   #0f172a;  /* Dark navy for data tables */
  --color-accent:      #f59e0b;  /* Amber for highlights/alerts */
  --color-background:  #f9fafb;
  --color-surface:     #ffffff;
  --font-display:      'Inter', 'Noto Sans', sans-serif;
  --font-body:         'Inter', sans-serif;
  --radius-default:    8px;      /* Precise, professional */
}

/* BUYER INTELLIGENCE TOKENS */
:root[data-app="buyer"] {
  --color-primary:     #22c55e;   /* Bright green on dark */
  --color-secondary:   #3b82f6;   /* Data blue */
  --color-accent:      #f59e0b;   /* Alert amber */
  --color-background:  #0f1117;   /* Deep dark */
  --color-surface:     #1a1f2e;   /* Card surface */
  --color-surface-2:   #242b3d;   /* Nested surface */
  --color-text:        #e2e8f0;   /* Off-white text */
  --color-text-muted:  #94a3b8;
  --font-display:      'Space Grotesk', sans-serif;
  --font-body:         'DM Sans', sans-serif;
  --font-mono:         'JetBrains Mono', monospace; /* For numbers/data */
  --radius-default:    6px;       /* Tight — efficient */
}
```

16.3 Farmer App Component Patterns

```
NAVIGATION: Bottom tab (5 tabs with large icons + vernacular labels)
CARDS: Full-width, 16px padding, clear photo + crop name + status chip
INPUTS: Large, 60px height, voice input button prominent
BUTTONS: Full-width primary CTAs; 56px height; high contrast
ALERTS: Color-coded top banner (red=urgent, yellow=warning, blue=info)
PHOTOS: Camera-first; guide overlay (crop needs to be in frame)
PROGRESS: Large checkmarks, completion percentage, encouraging copy in Telugu
EMPTY STATES: Illustrated (crop illustrations); voice-prompted action
```

SECTION 17 — ANALYTICS & INTELLIGENCE REPORTING

17.1 Platform KPIs (Internal Admin Dashboard)

```
Farmer Metrics:
  - Total registered farmers (by state, district)
  - Active declarations (current season)
  - Data quality avg score
  - DAU/MAU ratio (engagement)
  - Crop declaration → availability post rate
  - Inquiry response rate (for farmer-consent users)

FPO Metrics:
  - Active FPOs (by subscription tier)
  - Total member farmers in system
  - Monthly procurement volume (tons)
  - Supply listings created vs. expired
  - Buyer inquiry response rate
  - MRR (Monthly Recurring Revenue) by plan

Buyer Metrics:
  - Active buyers (by plan)
  - Search sessions per day
  - Inquiry send rate
  - Inquiry-to-deal conversion (self-reported)
  - Feature usage: heatmap / forecast / trend (engagement)
  - Renewal rate by plan tier

Intelligence Accuracy:
```

```
  - Declaration yield vs. actual yield (where known)
  - Forecast accuracy (30-day): MAE, MAPE
  - Data coverage: % of districts with >10 farmer declarations
```

17.2 Government Intelligence Package (Licensing)

```
Delivered: Monthly PDF + API feed

Contents:
  - District-wise crop production estimates (current season)
  - Harvest calendar (state-level, 90-day window)
  - Price trend correlation (supply vs. Agmarknet price)
  - Surplus/deficit alerts (early warning system)
  - FPO performance summary (for SFAC/NABARD)
  - Crop-specific supply chain analysis (for policy)

Format: PDF dashboard + downloadable Excel + API endpoint
Pricing: ₹50L – ₹5 Crores/year per state (or national)
```

SECTION 18 — THIRD-PARTY INTEGRATIONS

18.1 Government API Integrations (All Free)

API	Provider	Purpose	Endpoint
Agmarknet Price Data	AGMARKNET (GoI)	Daily APMC prices for 500+ crops	agmarknet.gov.in API
eNAM Market Data	eNAM Portal	Electronic national agriculture market prices	enam.gov.in API
PM-KISAN Farmer Database	PM-KISAN	Cross-verify farmer identity	pmkisan.gov.in
Soil Health Card	DBT Agriculture	Soil data for farmer profile enrichment	soilhealth.dac.gov.in
IMD Weather API	India Met Dept	District-level rainfall/temperature	mausam.imd.gov.in
Bhuvan Maps	ISRO	Satellite imagery of farm land	bhuvan.nrsc.gov.in

API	Provider	Purpose	Endpoint
State Land Records	AP Dharani, MH Mahabhulekh	Verify farm survey numbers	State-specific APIs
GST Verification	GSTIN API	Verify buyer GST number	api.gst.gov.in
Village/LGD Master	LGDMS	Village → District → State mapping	lgdirectory.gov.in
SFAC FPO Directory	SFAC India	Verify FPO registration	sfac.in API
DigiLocker	MeitY	Farmer Aadhaar linking	digilocker.gov.in

18.2 Commercial Integrations

Service	Provider	Cost	Purpose
SMS	Fast2SMS / MSG91	₹0.08/SMS	OTP + alerts
WhatsApp	Meta WABA	1,000 free/month	Inquiry alerts, notifications
Email	Resend	3,000 free/month	Reports, receipts
Maps	MapBox	50K free/month	Location, search, heatmaps
Payments	Razorpay	2%/txn	Subscriptions
Push Notifications	Firebase FCM	Free	Mobile alerts
AI/LLM	OpenAI	Pay-per-use	Summaries, auto-translate
Translation	LibreTranslate	Self-hosted free	Telugu/Hindi/Kannada UI strings
File Storage	Cloudflare R2	10GB free	Photos, documents, reports

---

SECTION 19 — NON-FUNCTIONAL REQUIREMENTS

19.1 Performance Targets

Metric	Target	Tool
Farmer app cold start (Android)	< 3 seconds on 4G	Detox

Metric	Target	Tool
Farmer app cold start (2G)	< 8 seconds	Manual testing
Supply search API (p95)	< 400ms	Supabase dashboard
Intelligence dashboard load	< 2 seconds	Lighthouse
Map heatmap render (100K points)	< 1.5 seconds	MapBox + WebGL
Procurement record save	< 500ms	Custom metric
Push notification delivery	< 5 seconds after trigger	FCM dashboard
File upload (5 photos)	< 4 seconds on 4G	Custom metric

19.2 Availability

```
Uptime target: 99.9% (8.7 hours downtime/year)
Critical path availability (farmer data entry): 99.95%

Offline resilience:
  - Farmer app: Core features work offline (as specified in Section 12.3)
  - FPO platform: Procurement recording cacheable for 30 min offline
  - Buyer app: Last search results cached for offline viewing

Disaster recovery:
  - Supabase PITR (Point-in-Time Recovery): 7 days
  - Cloudflare R2: Redundant storage (3 copies)
  - Maximum RPO (Recovery Point Objective): 1 hour
  - Maximum RTO (Recovery Time Objective): 2 hours
```

19.3 Scalability Design

```
Database scaling path:
  Year 1 (1M declarations): Supabase Free → Pro ($25/mo)
  Year 2 (10M declarations): Supabase Pro with read replica ($75/mo)
  Year 3 (50M declarations): Supabase Team + TimescaleDB for time-series ($200/mo)
  Year 5 (500M declarations): Partitioned PostgreSQL (by year × state)

Intelligence query optimization:
  - supply_intelligence table: Pre-computed aggregates (refreshed hourly)
  - GeoJSON heatmap: Cached in Upstash Redis (15-min TTL)
```

```
  - Price trends: Cached from Agmarknet (4-hour TTL)
  - FPO supply listings: Full-text indexed + cached (5-min TTL)

Mobile app performance:
  - Farmer app APK: Target < 25MB (critical for low-storage Android devices)
  - Image loading: Progressive (low-res blur placeholder → full resolution)
  - Offline SQLite: Max 50MB local storage per device
```

SECTION 20 — COMPLIANCE & LEGAL FRAMEWORK

20.1 Data Protection (DPDP Act 2023)

```
Data Fiduciary: AgriFlow India Pvt Ltd
Data Principal Rights:
  ✅ Right to access: "Download my data" → JSON export
  ✅ Right to correction: Edit all profile fields, re-upload documents
  ✅ Right to erasure: "Delete account" → 30-day soft delete → purge
  ✅ Right to grievance: grievance@agriflow.in, response within 48 hours
  ✅ Consent management: Granular per-data-type consent

Data Retention:
  - Active farmer profile: While active + 6 months post-deletion
  - Crop declarations: 5 years (agricultural data of public interest)
  - Procurement records: 7 years (tax compliance)
  - Payment records: 7 years (Income Tax Act)
  - Community posts: 1 year or until deleted by user
  - Server logs: 30 days

Consent for marketing:
  - Separate opt-in (not bundled with ToS)
  - Withdrawal: Instant, one-tap in settings
```

20.2 Agriculture-Specific Compliance

```
Farmer Privacy (Agricultural Produce Markets Act):
  - No farmer-level data sold to competing aggregators
  - No farmer data shared with input companies (pesticide, seed) without consent
  - FPO cannot export farmer data outside AgriFlow without farmer consent

RERA Not Applicable: AgriFlow is not a real estate platform
```

```
e-NAM Compliance:
  - Supply listings must not misrepresent quantities
  - Price expectations are indicative only (not binding offers)
  - Platform does not facilitate price fixing or cartelization

SFAC / NABARD Reporting:
  - FPOs who opt-in can authorize AgriFlow to share aggregated data with
SFAC/NABARD
  - This is a service feature (not mandatory)
  - Data shared: procurement volumes, member count, crop types (never individual
farmer data)
```

SECTION 21 — DEVELOPER EXECUTION PLAN

21.1 Team Structure

Role	Count	Primary Focus
Full-Stack Lead	1	Next.js (FPO + Buyer), Supabase
Mobile Developer	1	React Native (Expo) — Farmer App
Backend / Data Engineer	1	Edge Functions, intelligence pipeline, government APIs
UI/UX Designer	1	Three distinct design systems
DevOps (part-time)	0.5	Vercel, GitHub Actions, monitoring
QA Engineer	1	Playwright (web), Detox (mobile)
Data Science (part-time)	0.5	Supply forecasting models
Field Operations	2	Farmer onboarding agents (non-tech; Phase 1 launch)

21.2 Phase-by-Phase Execution Plan

Phase 0: Infrastructure & Foundation (Weeks 1–2)

```
GOAL: Dev environment, CI/CD, auth, base schema


Tasks:
```

```
  [ ] Initialize Next.js 14 (FPO + Buyer web — monorepo)
  [ ] Initialize Expo React Native (Farmer App)
  [ ] Set up Supabase (DB + Auth + Realtime + Storage)
  [ ] Create Cloudflare R2 bucket + CDN
  [ ] Configure Vercel deployment + GitHub Actions CI/CD
  [ ] Implement Supabase phone OTP authentication
  [ ] Apply full database migration (all tables from schema above)
  [ ] Set up RLS policies for all tables
  [ ] Configure Sentry + PostHog
  [ ] Set up environment variables (all services)
  [ ] Seed: crops master table (200+ crops with Telugu/Hindi names)
  [ ] Seed: village/district/state master (LGD data)
  [ ] Load test baseline (k6 against Supabase)

DELIVERABLE: Auth flow working across all 3 apps
```

Phase 1: Farmer App — Core (Weeks 3-7)

```
GOAL: Farmer can register, declare crop, post availability, see community feed

Tasks:
  [ ] Farmer onboarding: phone + OTP + profile (village auto-detect)
  [ ] Farm details: land, irrigation, primary crops
  [ ] Crop declaration: crop select (vernacular + visual) + area + dates + yield
  [ ] Harvest calendar view (crop countdown)
  [ ] Availability posting (near harvest)
  [ ] Community feed: view + post + upvote
  [ ] Offline sync: expo-sqlite + pending queue
  [ ] Voice input: Telugu/Hindi crop name recognition
  [ ] Push notifications: FCM integration
  [ ] Photo upload: Cloudflare R2 via pre-signed URLs
  [ ] Full Telugu UI (all strings externalized + translated)
  [ ] Farmer dashboard (active crops, inquiries)
  [ ] Privacy settings (contact consent levels)

DELIVERABLE: Farmer app on TestFlight (iOS) + APK (Android beta)
```

Phase 2: FPO Management Platform — Core (Weeks 8-13)

```
GOAL: FPO can manage farmers, record procurement, track inventory

Tasks:
  [ ] FPO registration + document upload + verification flow
```

```
  [ ] FPO dashboard (member count, procurement summary, pending payments)
  [ ] Member farmer management (add, bulk import, QR join)
  [ ] Individual member profile (crops, procurement history, payments)
  [ ] Procurement recording: farmer select → crop → qty → quality → price →
receipt
  [ ] Receipt generation (PDF via React-PDF, shareable via WhatsApp)
  [ ] Inventory management: auto-update from procurement + manual adjustment
  [ ] Inventory freshness alerts (configurable by crop)
  [ ] Farmer payment management: payment queue, batch record, payment history
  [ ] FPO supply listing creation (from inventory)
  [ ] Analytics dashboard: procurement summary, payment status
  [ ] Exportable reports: PDF + CSV
  [ ] WhatsApp receipt sending (Meta WABA integration)
  [ ] Multi-staff access: FPO admin + field staff roles

DELIVERABLE: FPO platform fully functional (web + mobile tab)
```

Phase 3: Supply Intelligence Data Engine (Weeks 14–16)

```
GOAL: Central intelligence engine computing district-level supply aggregates

Tasks:
  [ ] supply_intelligence table population pipeline (Trigger.dev batch job)
  [ ] District-level aggregation logic (k-anonymity enforcement)
  [ ] Hourly refresh: new declarations → update aggregates
  [ ] Agmarknet API integration: fetch and cache daily prices
  [ ] IMD weather API: district-level daily weather
  [ ] eNAM market price integration
  [ ] Data quality scoring pipeline
  [ ] Harvest forecast computation (statistical model v1: area × yield × weather
factor)
  [ ] Anomaly detection: flag outlier declarations for review
  [ ] Supply intelligence API endpoints
  [ ] Admin intelligence dashboard (internal)

DELIVERABLE: Live district-level supply intelligence for 5 pilot states
```

Phase 4: Buyer Intelligence App (Weeks 17–21)

```
GOAL: Buyers can search supply, view intelligence, send inquiries

Tasks:
  [ ] Buyer registration + GST verification
```

```
  [ ] Plan selection + Razorpay subscription payment
  [ ] Supply search engine: crop + state + district + harvest window + quantity +
grade filters
  [ ] Search result cards: FPO listings + aggregated district supply
  [ ] FPO supply listing detail view (full info, FPO profile)
  [ ] District aggregate supply view (k-anonymized)
  [ ] Buyer inquiry: send inquiry to FPO / district
  [ ] FPO inquiry inbox (FPO side of inquiry management)
  [ ] Watchlist: create alerts (crop × region × threshold)
  [ ] Alert system: push + email + WhatsApp triggers
  [ ] Intelligence reports: supply heatmap (MapBox choropleth)
  [ ] Intelligence reports: 30-day harvest forecast chart
  [ ] Intelligence reports: price trend overlays
  [ ] FPO directory (enterprise plan)
  [ ] Enterprise: API key management + rate-limited data API
  [ ] Buyer dashboard: inquiry history, watchlist, saved FPOs

DELIVERABLE: Buyer app live; first paying buyers onboarded
```

Phase 5: Farmer Inquiry Flow + KYC (Weeks 22–24)

```
GOAL: Complete data loop — farmer receives inquiries; KYC integration

Tasks:
  [ ] Farmer inquiry notification: push + SMS
  [ ] Farmer inquiry inbox: view inquiry details, express interest
  [ ] Consent-based contact routing logic
  [ ] Aadhaar e-KYC integration (DigiLocker)
  [ ] PM-KISAN farmer verification integration
  [ ] KYC level upgrade flow (phone → Aadhaar → document)
  [ ] Verified badge display on farmer profile + availability listing
  [ ] Data quality score update post-KYC
  [ ] FPO-verified flag for FPO-confirmed crop declarations

DELIVERABLE: Farmer → Buyer inquiry loop completed; KYC live
```

Phase 6: Government Integrations + Financial Services (Weeks 25–28)

```
GOAL: Government API integrations; financial services referral

Tasks:
  [ ] State land records API integration (AP Dharani, MH Mahabhulekh)
  [ ] Soil Health Card data integration
```

```
  [ ] PM-KISAN data linkage (farmer verification)
  [ ] SFAC FPO directory verification
  [ ] Government intelligence report generation (PDF + API)
  [ ] NABARD / State Agri Dept partnership dashboard
  [ ] Crop loan eligibility score (based on crop declaration + land data)
  [ ] Crop insurance integration (PMFBY facilitation)
  [ ] Financial partner referral flow (NBFC / SFB crop loan links)
  [ ] Agronomist advisory marketplace (Phase 2 revenue stream)

DELIVERABLE: Government data integrations live; first institutional contracts signed
```

Phase 7: Intelligence Upgrade + Scale (Weeks 29–34)

```
GOAL: ML-powered forecasting; national scale; data licensing

Tasks:
  [ ] ML supply forecast model (XGBoost) — trained on 2 seasons of data
  [ ] Model deployment (Railway free tier Python microservice)
  [ ] Forecast accuracy dashboard (internal)
  [ ] Crop production heatmap (district-level choropleth, animated)
  [ ] Supply-demand gap alerts (daily computation)
  [ ] National supply overview dashboard (enterprise)
  [ ] Demand signal tracking (buyer inquiry patterns → demand intelligence)
  [ ] Data licensing API (rate-limited, key-managed, institutional pricing)
  [ ] Season-end accuracy report (forecast vs. actuals)
  [ ] Expand to 15 states (north: Punjab, Haryana, UP; south: TN, Kerala)
  [ ] Vernacular expansion: Marathi, Punjabi, Tamil
  [ ] Load testing: 1M concurrent users (k6)

DELIVERABLE: India's most comprehensive crop supply intelligence platform
```

SECTION 22 — AI PROMPT TEMPLATES FOR EACH MODULE

Prompt 1: Farmer App — Vernacular Crop Declaration

```
Build a crop declaration UI component for a React Native (Expo) farmer app
targeting semi-literate Telugu farmers.

Requirements:
- Language: Full Telugu UI (all labels, buttons, messages in Telugu)
```

```
- Crop selection: Visual grid of crop photos (200+ crops) with Telugu names
- Voice input: Microphone button on all text fields (React Native Voice)
- Area input: Slider + number; unit toggle (acres/cents/guntas/bigha)
- Date picker: Planting date + harvest date (native date picker)
- Yield input: Number + unit (kg/quintal/ton)
- Field: Variety (optional text field with voice)
- Summary screen: Large text, crop photo, key figures — confirm before save
- Offline: Save to expo-sqlite if no connection; sync when online
- Accessibility: Minimum 60px tap targets; high contrast; 24px minimum font

Technical stack: React Native (Expo SDK 51), TypeScript, Zustand, expo-sqlite,
expo-speech, MMKV

Deliver:
- CropDeclarationForm component (multi-step with progress bar)
- VoiceInput wrapper component (wraps any TextInput with voice button)
- CropSelector component (visual grid, searchable in Telugu, paginated)
- OfflineQueue utility (expo-sqlite FIFO queue for unsynced declarations)
- Unit conversion utilities (acres ↔ cents ↔ guntas ↔ bigha)
- i18n strings for Telugu (all UI copy)
- Jest tests for unit conversion + queue logic
```

Prompt 2: FPO Platform — Procurement Recording

```
Build a full procurement recording module for an FPO management web application.

Stack: Next.js 14 (App Router), TypeScript, Supabase, React Hook Form + Zod,
shadcn/ui, TanStack Table

Requirements:
- Select farmer: Search by name or phone; QR scan option (camera via browser)
- Auto-populate: If farmer has active crop declarations, show crop chips to pre-
fill
- Quantity: Number input + unit (kg/quintal/ton); real-time conversion shown
- Quality grade: Premium / Grade A / Grade B / Reject (radio with color coding)
- Moisture content: Optional numeric input (%)
- Procurement price: Per unit (₹); editable
- Auto-calculate: Gross amount = qty × price; deductions; net payable
- Generate receipt: PDF (React-PDF) with FPO logo, farmer name, crop, qty, price,
net payable, date, signature line
- WhatsApp share: Receipt shared as PDF to farmer's WhatsApp (Meta WABA API)
- Inventory auto-update: On procurement save → add to FPO inventory
- Farmer payment queue: Create pending payment entry with net payable
```

```
Deliver:
- ProcurementForm component (multi-step: farmer select → crop details → pricing →
confirm)
- ReceiptTemplate component (printable, React-PDF)
- WhatsApp API integration function (send PDF as document)
- Supabase edge function: create procurement + update inventory (atomic
transaction)
- Zod schema: Full validation of procurement data
- TanStack Table: Procurement history view with filters
- Playwright E2E test: Full procurement flow
```

Prompt 3: Supply Intelligence Engine

```
Build the central supply intelligence computation pipeline for an agri-tech
platform.

Context: Farmers declare crop plantings (crop, area, expected yield, harvest date,
location).
FPOs record actual procurement. The system must aggregate this into district-level
supply intelligence
that buyers can search, with k-anonymity (never expose individual farmer data;
minimum k=5 farmers per cell).

Stack: TypeScript, Supabase Edge Functions (Deno), Trigger.dev (cron jobs),
Upstash Redis (cache)

Requirements:
1. Aggregation computation (run hourly):
   - Group crop_declarations by (crop_code × state × district × harvest_window)
   - Harvest window: bucket into: now, 1-7d, 8-14d, 15-30d, 31-60d, 61-90d
   - Compute: total_area_acres, total_yield_tons (expected), farmer_count
   - Apply k-anonymity: if farmer_count < 5, set below_threshold = true (don't
show count)
   - Quality distribution: % of each grade
   - Upsert into supply_intelligence table

2. Agmarknet price fetch (run daily):
   - Fetch latest APMC modal price for each crop × district pair
   - Cache in Upstash Redis (4-hour TTL)
   - Compute 7-day and 30-day price trend (% change)
   - Update supply_intelligence.price_agmarknet and trend fields
```

```
3. Buyer alert dispatch (run hourly):
   - For each buyer_watchlist: check if new supply matches criteria
   - If new match found: push notification (FCM) + WhatsApp alert
   - Rate limit: Max 3 alerts/day per watchlist

4. Data quality pipeline (run daily):
   - Score each declaration: yield per acre vs. district average, GPS validity,
FPO verification
   - Flag outliers for admin review
   - Exclude low-quality data from intelligence aggregates

Deliver:
- Trigger.dev job: aggregation_pipeline (hourly)
- Trigger.dev job: price_fetch (daily, Agmarknet API)
- Trigger.dev job: buyer_alert_dispatch (hourly)
- Trigger.dev job: data_quality_score (nightly)
- Supabase Edge Function: /intelligence/district/:district/crop/:crop (cached
query)
- Redis caching layer (Upstash client)
- Unit tests: k-anonymity logic, aggregation math, price trend computation
```

Prompt 4: Buyer Supply Search Engine

```
Build a high-performance supply search engine for agri buyers.

Stack: Next.js 14, TypeScript, Supabase (PostGIS + FTS), Upstash Redis, MapBox
(react-map-gl)

Requirements:
- Full-text search: Crop name, FPO name, district (Supabase FTS with tsvector)
- Geospatial filter: Supply within radius of buyer's location (PostGIS ST_DWithin)
- Filters: crop, state(s), district(s), harvest window, quantity min, quality
grade, certification, source type (FPO/aggregate)
- Sort: relevance / harvest date / quantity / last updated
- Pagination: Cursor-based (no offset for performance)
- Result cards: FPO supply listings + aggregated district-level supply
- Map view: MapBox choropleth heatmap + supply listing pins
- Caching: Cache identical queries in Upstash Redis (5-min TTL)
- Plan gating: Explorer plan → 5 states; Trader plan → 10 states; Enterprise →
all-India

Performance target: p95 < 400ms for search query
```

```
Deliver:
- SupplySearchBar component (crop typeahead + voice)
- FilterPanel component (desktop sidebar + mobile bottom sheet)
- SearchResults component (list view + map view toggle)
- SupplyCard component (FPO listing variant + aggregate variant)
- SupplyMapView component (MapBox choropleth + pins)
- Supabase Edge Function: /supply/search (with RLS + plan gating)
- Redis caching middleware
- SQL: Optimized search query (EXPLAIN ANALYZE verified < 200ms)
- Playwright tests: Search with all filter combinations
```

SECTION 23 — RISK REGISTER & EDGE CASES

23.1 Business Risks

Risk	Probability	Impact	Mitigation
Farmer non-adoption (too complex)	High	Critical	Voice-first, visual-first, FPO-led onboarding, field agents
Data quality — inflated declarations	High	High	Data quality scoring, FPO verification, actual vs. declared tracking
FPO data entry resistance (manual habit)	Medium	High	Simple mobile UI; WhatsApp receipt trigger as hook; FSM training
Buyer unwilling to pay before network effect	Medium	High	14-day free trial; 2-state access on Explorer; pilot with 5 large buyers
Government API downtime (Agmarknet, IMD)	Medium	Medium	Cache last known data (4h–24h); degrade gracefully with "data may be delayed"
FPO sharing farmer contact without consent	Medium	High	RLS policies; audit log; FPO ToS with penalty clause
Competitor (DeHaat, AgroStar) adding supply intel	Low	High	Speed to data; government partnerships as moat; vernacular as differentiator
Political sensitivity of agriculture data	Low	High	No price prediction (only historical data + declared forecast); no trading

Risk	Probability	Impact	Mitigation
DPDP Act non-compliance	Low	Critical	Privacy-by-design; DPA engagement early; legal counsel before launch

23.2 Technical Edge Cases

```
Edge Case 1: Farmer declares same crop + field twice
  Detection: Within 7 days, same farmer + same crop + same approximate location
  Action: "You already have a tomato declaration — do you want to update it
instead?"
  Resolution: Merge prompt; if different field, allow with "New plot" confirmation

Edge Case 2: FPO records procurement but farmer's declaration doesn't match
  (e.g., farmer declared 5 tons, FPO procures 12 tons)
  Detection: procurement_qty > declaration_expected_yield × 1.5
  Action: Alert FPO staff: "This exceeds the farmer's declared yield. Please
confirm."
  Resolution: FPO confirms → actual_yield updated; or data quality flag raised

Edge Case 3: Village name collision (same name, different districts)
  Resolution: Always store village + mandal + district + state combo; LGD village
code as unique key
  Display: "Palavalasa, Nandigama, Krishna, AP" — never just "Palavalasa"

Edge Case 4: Farmer changes harvest date repeatedly
  Detection: > 3 date changes on same declaration
  Action: Note added to declaration: "Harvest date revised 4 times" — reduces data
quality score
  Action: FPO notified (for their member farmers) to verify

Edge Case 5: Buyer sends mass inquiries (spam)
  Detection: > 20 inquiries/day from one buyer account
  Action: Rate limit triggered; account flagged; support team review
  FPO impact: Multiple simultaneous inquiries from same buyer shown as one

Edge Case 6: FPO lists more supply than their recorded inventory
  Detection: supply_listing.quantity > sum(fpo_inventory.quantity) for that crop
  Action: Warning shown during listing creation: "Your inventory shows only X
tons"
  Resolution: FPO confirms or updates — forecasted supply can exceed current
inventory
```

```
Edge Case 7: Farmer moves to different district
  Detection: GPS location in new declaration significantly differs from profile
  Action: "Looks like you're in a different location. Update your profile?" prompt
  Resolution: Profile updated; old declarations retain original location

Edge Case 8: FPO dissolution or fraudulent FPO
  Detection: Registration number invalid on SFAC cross-check
  Action: FPO suspended; all supply listings unpublished; member farmers notified
  Farmer impact: Farmers can continue using app independently; choose new FPO

Edge Case 9: Price manipulation (FPO posts unrealistic price expectation)
  Detection: Listed price > 3× Agmarknet modal price for same crop × district
  Action: Warning during listing creation; flagged for admin review
  Display: "Price range significantly higher than current market — verify this is correct"

Edge Case 10: Community post spreading misinformation about crop disease
  Detection: Post flagged by 5+ users OR AI model detects false agricultural claim
  Action: Post hidden pending review; author notified
  Resolution: Verified agronomist (partnered) reviews within 24 hours → restore or remove
```

SECTION 24 — FUTURE INTELLIGENCE ROADMAP

Phase 3 Intelligence Features (Months 10–18)

```
Feature 1: Crop Production Heatmaps
  - India district map with crop intensity overlays
  - Filter: crop × date × quality
  - Animated: season progression from planting → harvest
  - Use case: Buyer identifies sourcing regions; Govt tracks production concentration

Feature 2: AI Supply Forecasting (ML Model)
  - Input: Declarations + historical actuals + weather + satellite NDVI data
  - Output: 30/60/90-day production forecast by crop × district
  - Confidence intervals based on data density
  - Season-over-season comparison

Feature 3: Demand Signals Layer
  - Aggregate buyer inquiry patterns → demand heat by crop × region
```

```
  - Supply-demand gap alerts: "Tomato demand exceeds supply by 30% in AP this week"
  - Price forecast signal (supply + demand → price direction probability)

Feature 4: Farmer Decision Intelligence
  - Personalized crop calendar: "Based on your soil and region, plant Rabi onion by Nov 15"
  - Price outlook when declaring crop: "Tomato supply in your district expected to peak — consider staggering harvest"
  - Risk alert: "Weather forecast indicates frost risk in your district — protect chilli seedlings"

Feature 5: NDVI Satellite Monitoring
  - Integrate ISRO Bhuvan NDVI data
  - Validate farmer declarations (crop health proxy)
  - Auto-alert: "Crop stress detected in your farm area"
  - Use case: Crop insurance trigger; loan assessment

Feature 6: Agricultural Finance Layer
  - Farmer credit score (based on crop declaration accuracy, FPO membership, land records, PM-KISAN)
  - Pre-approved crop loan offer (partnered NBFC/SFB)
  - PMFBY crop insurance enrollment (digital, in-app)
  - Input credit (seed, fertilizer) against harvest receivable
```

SECTION 25 — APPENDIX

Appendix A: Enum Type Definitions

```sql
```

```
CREATE TYPE user_role_enum AS ENUM ('farmer', 'fpo_admin', 'fpo_staff', 'buyer', 'fi
CREATE TYPE language_enum AS ENUM ('telugu', 'hindi', 'kannada', 'marathi', 'punjabi
CREATE TYPE farming_method_enum AS ENUM ('conventional', 'organic', 'natural_farming
CREATE TYPE contact_consent_enum AS ENUM ('no_contact', 'fpo_only', 'verified_buyers
CREATE TYPE fpo_type_enum AS ENUM ('fpo', 'fpc', 'cooperative', 'pacs', 'vco', 'agri
CREATE TYPE verification_status_enum AS ENUM ('pending', 'in_review', 'verified', 'r
CREATE TYPE crop_category_enum AS ENUM ('vegetables', 'fruits', 'cereals', 'pulses',
CREATE TYPE declaration_status_enum AS ENUM ('declared', 'growing', 'available', 'pa
CREATE TYPE quality_grade_enum AS ENUM ('premium', 'grade_a', 'grade_b', 'grade_c',
CREATE TYPE listing_visibility_enum AS ENUM ('all_buyers', 'verified_buyers', 'fpo_m
CREATE TYPE buyer_type_enum AS ENUM ('trader', 'exporter', 'food_processor', 'retail
CREATE TYPE buyer_plan_enum AS ENUM ('trial', 'explorer', 'trader', 'enterprise', 'i
CREATE TYPE inquiry_status_enum AS ENUM ('sent', 'delivered', 'read', 'replied', 'de
CREATE TYPE payment_status_enum AS ENUM ('pending', 'initiated', 'captured', 'settle
CREATE TYPE sub_status_enum AS ENUM ('trial', 'active', 'expired', 'cancelled', 'pau
CREATE TYPE post_type_enum AS ENUM ('pest_alert', 'weather_alert', 'irrigation_tip',
CREATE TYPE ticket_category_enum AS ENUM ('data_issue', 'payment_issue', 'account_is
CREATE TYPE ticket_priority_enum AS ENUM ('low', 'medium', 'high', 'urgent');
CREATE TYPE ticket_status_enum AS ENUM ('open', 'assigned', 'in_progress', 'waiting_
```

Appendix B: Environment Variables

```
env
```

```
# Supabase
SUPABASE_URL=
SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=      # Server-only, never expose

# Cloudflare
CF_ACCOUNT_ID=
CF_R2_BUCKET_NAME=
CF_R2_ACCESS_KEY_ID=
CF_R2_SECRET_ACCESS_KEY=
CF_R2_PUBLIC_URL=               # CDN URL for assets

# MapBox
NEXT_PUBLIC_MAPBOX_TOKEN=

# Razorpay
RAZORPAY_KEY_ID=                # Public (client-safe)
RAZORPAY_KEY_SECRET=            # Server-only
RAZORPAY_WEBHOOK_SECRET=        # Webhook signature verification

# SMS
FAST2SMS_API_KEY=
MSG91_AUTH_KEY=
MSG91_SENDER_ID=

# WhatsApp (Meta WABA)
META_WABA_TOKEN=
META_WABA_PHONE_NUMBER_ID=
META_WABA_VERIFY_TOKEN=         # Webhook verification

# Email
RESEND_API_KEY=

# Push Notifications
FIREBASE_SERVICE_ACCOUNT_JSON=  # FCM server-side

# Government APIs
AGMARKNET_API_KEY=
ENAM_API_KEY=
IMD_API_KEY=
LGDMS_API_KEY=
DIGILOCKER_CLIENT_ID=
DIGILOCKER_CLIENT_SECRET=
```

```
SFAC_API_KEY=

# OpenAI (AI features)
OPENAI_API_KEY=

# Background Jobs
TRIGGER_API_KEY=

# Cache
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=
UPSTASH_QSTASH_URL=
UPSTASH_QSTASH_TOKEN=

# Monitoring
SENTRY_DSN=
SENTRY_AUTH_TOKEN=

# Analytics
NEXT_PUBLIC_POSTHOG_KEY=
NEXT_PUBLIC_POSTHOG_HOST=

# App
NEXT_PUBLIC_APP_URL=https://agriflow.in
NEXT_PUBLIC_FARMER_APP_DEEPLINK=agriflow://
ENCRYPTION_KEY=                 # AES-256 for bank account encryption
NODE_ENV=production
```

Appendix C: Folder Structure

```
agriflow/
├── apps/
│   ├── web-fpo/                     # FPO Management Platform (Next.js 14)
│   │   ├── app/
│   │   │   ├── (auth)/              # Login, register FPO
│   │   │   ├── dashboard/           # Main FPO dashboard
│   │   │   ├── members/             # Farmer member management
│   │   │   ├── procurement/         # Procurement recording
│   │   │   ├── inventory/           # Inventory tracking
│   │   │   ├── supply-listings/     # Create + manage supply listings
│   │   │   ├── inquiries/           # Buyer inquiry inbox
│   │   │   ├── payments/            # Farmer payment management
│   │   │   ├── reports/             # Analytics + export
```

```
│   │   │   └── api/                   # API routes (BFF)
│   │   └── components/
│   │       ├── ui/                    # shadcn/ui + custom atoms
│   │       ├── procurement/           # Procurement-specific components
│   │       ├── inventory/             # Inventory components
│   │       └── charts/                # Analytics charts
│   │
│   ├── web-buyer/                     # Buyer Intelligence App (Next.js 14)
│   │   ├── app/
│   │   │   ├── (auth)/
│   │   │   ├── dashboard/
│   │   │   ├── search/                # Supply search engine
│   │   │   ├── intelligence/          # Reports, heatmaps, forecasts
│   │   │   ├── inquiries/             # Sent inquiries + responses
│   │   │   ├── watchlists/
│   │   │   ├── fpo-directory/         # Enterprise FPO directory
│   │   │   ├── account/               # Plan, billing, API keys
│   │   │   └── api/
│   │   └── components/
│   │       ├── ui/                    # Buyer dark-theme design system
│   │       ├── supply/                # Supply search components
│   │       ├── intelligence/          # Charts, heatmaps, forecasts
│   │       └── maps/                  # MapBox components
│   │
│   └── mobile-farmer/                 # Farmer App (Expo React Native)
│       ├── app/                       # Expo Router
│       │   ├── (auth)/
│       │   ├── (tabs)/
│       │   │   ├── home.tsx
│       │   │   ├── crops.tsx          # My crops + declarations
│       │   │   ├── availability.tsx   # Post availability
│       │   │   ├── community.tsx      # Feed
│       │   │   └── profile.tsx
│       │   └── declaration/           # Crop declaration multi-step
│       ├── components/
│       │   ├── crop/                  # CropSelector, CropCard
│       │   ├── voice/                 # VoiceInput wrapper
│       │   ├── forms/                 # Form atoms for farmer UI
│       │   └── feed/                  # Community feed components
│       └── lib/
│           ├── offline/               # expo-sqlite queue
│           ├── i18n/                  # Telugu/Hindi/Kannada strings
│           └── voice/                 # Voice recognition utilities
│
```

```
├── packages/
│   ├── shared-types/                   # TypeScript interfaces (all apps share)
│   ├── shared-zod/                     # Zod validation schemas (shared)
│   ├── shared-utils/                   # Common pure utilities
│   └── crop-master/                    # Crop database (JSON + types)
│
├── supabase/
│   ├── migrations/                     # Ordered SQL migrations
│   │   ├── 001_initial_schema.sql
│   │   ├── 002_rls_policies.sql
│   │   ├── 003_indexes.sql
│   │   ├── 004_functions.sql           # PostgreSQL functions
│   │   └── 005_triggers.sql            # DB triggers
│   ├── functions/                      # Supabase Edge Functions (Deno)
│   │   ├── auth/                       # OTP, session
│   │   ├── search/                     # Supply search engine
│   │   ├── intelligence/               # Intelligence API
│   │   ├── webhooks/                   # Razorpay, Meta WABA
│   │   ├── notifications/              # Alert dispatch
│   │   └── government/                 # Agmarknet, IMD integrations
│   └── seed/                           # Crop master, village master, test data
│
├── jobs/                               # Trigger.dev background jobs
│   ├── aggregation-pipeline.ts         # Hourly supply aggregation
│   ├── price-fetch.ts                  # Daily Agmarknet price fetch
│   ├── buyer-alert-dispatch.ts         # Watchlist alert delivery
│   ├── data-quality-score.ts           # Nightly quality scoring
│   └── harvest-reminders.ts            # T-15 day harvest reminder to farmers
│
├── docs/
│   ├── api/                            # API documentation (OpenAPI spec)
│   ├── architecture/                   # Architecture decision records
│   └── government/                     # Government API integration docs
│
├── .github/
│   └── workflows/
│       ├── ci.yml                      # Test + build on PR
│       ├── deploy-web.yml              # Vercel deploy on merge to main
│       └── deploy-edge.yml             # Supabase functions deploy
│
└── README.md
```

Appendix D: Launch Checklist

```
Pre-Launch (Week -3):
  [ ] Supabase production project created (separate from dev)
  [ ] All RLS policies tested on production schema
  [ ] Cloudflare R2 bucket configured (production)
  [ ] Domain: agriflow.in registered + SSL configured + Cloudflare proxy
  [ ] Farmer app: Google Play Console account + app submitted for review
  [ ] Farmer app: Apple Developer account + app submitted for review
  [ ] Razorpay KYC completed + bank account for settlements
  [ ] Fast2SMS DLT registration (mandatory for transactional SMS in India)
  [ ] Meta WABA: Business verification completed; templates approved
  [ ] DPDP Act: Privacy policy + ToS reviewed by India-qualified lawyer
  [ ] Grievance officer designated; contact published on website
  [ ] GST registration completed (if revenue expected in Year 1)
  [ ] Government API access: Agmarknet, IMD, LGDMS — keys obtained
  [ ] Pilot FPOs: 5 FPOs in AP/Karnataka agreed to participate
  [ ] Pilot buyers: 3 traders agreed to test buyer platform
  [ ] Field agents (2): Trained on farmer onboarding process
  [ ] Security: OWASP ZAP scan on all API endpoints
  [ ] Load test: k6 — 10,000 concurrent users on search API
  [ ] Smoke test: Full farmer → FPO → buyer flow end-to-end

Launch Day:
  [ ] Database: Seed crop master (200+ crops) + village master
  [ ] Feature flags: Disable Phase 2 features not yet built
  [ ] Monitoring: Sentry + PostHog fully configured + alert thresholds set
  [ ] On-call: Who handles P0 incidents (page policy defined)
  [ ] Farmer app: APK direct install link (pre-Play Store approval)
  [ ] WhatsApp broadcast: Pilot FPO farmers invited
  [ ] Press: AgriTech India + local Telugu press coverage
  [ ] Support: WhatsApp support number live + team briefed

Post-Launch (Month 1-3):
  [ ] Weekly review: Farmer declaration count, data quality scores
  [ ] Bi-weekly FPO calls: Usage, pain points, feature requests
  [ ] First buyer intelligence report: Sent to pilot buyers (manual PDF if needed)
  [ ] Government outreach: AP Agriculture Department demo scheduled
  [ ] NABARD SFAC partnership conversations initiated
  [ ] Data accuracy audit: 10 random declarations verified by field agents
  [ ] Play Store full public release (after TestFlight + APK beta validation)
  [ ] First media coverage: Hindu Business Line / Economic Times AgriTech
```

Appendix E: KPI Dashboard — Metrics That Matter

KPI	Definition	Target (Month 6)	Target (Month 18)
Registered Farmers	Total onboarded farmers	50,000	5,00,000
Active Declarations	Current season active crop declarations	20,000	2,00,000
Data Quality Avg	Average declaration quality score (0-100)	>65	>75
Districts Covered	Districts with ⩾10 active declarations	50	300
Active FPOs	FPOs with ⩾1 supply listing in last 30 days	100	1,500
FPO MRR	Monthly recurring revenue from FPO plans	₹5L	₹75L
Buyer Accounts	Registered and trial/paid buyers	200	2,000
Buyer MRR	Monthly recurring from buyer plans	₹3L	₹1Cr
Supply Searches/Day	Buyer supply search sessions	500	10,000
Inquiry Rate	Buyer inquiries sent per 100 search sessions	>8	>15
Forecast Accuracy	30-day harvest forecast MAPE	<25%	<15%
Farmer DAU/MAU	Ratio (engagement)	>20%	>35%
FPO Response Rate	% of buyer inquiries responded in 24h	>60%	>80%
Govt Partnerships	State agriculture departments using intelligence	1	5
Annual NPS (Farmer)	Net Promoter Score from farmer survey	>40	>55
Annual NPS (Buyer)	Net Promoter Score from buyer survey	>45	>60
Platform Gross Margin	Revenue minus direct costs	>80%	>85%

Document Version: 1.0 Classification: Confidential — Internal Use Only Prepared for: AgriFlow India Development Team Next Review: Quarterly or on major roadmap update

---

END OF DOCUMENT *AgriFlow India — From Field to Intelligence. India's Agriculture Supply Network.*